



October 6 University



Faculty of Information Systems
& Computer Sciences

Graduation Project

Rationalize the Consumption of Antibiotics System

Prepared By

Student Names

Department

Mohamed Said Abdel Fattah

CS

Ahmed Mohamed Ismail

CS

Rady Mohamed Rady

CS

Arafa Ahmed Salem

IS

Ali Ahmed Ali

IS

Alaa Abdullah Tawfik

IS

Supervised by

Dr. Eng. Samey Elmokadem

Dr. Abdelrahman El-Akhdar

Eng. Ahmed Rashad

June 2023

Abstract

Antibiotics are important drugs for the treatment of bacterial infections. However, their overuse and misuse have led to the emergence of antibiotic-resistant bacteria, which pose a significant threat to global public health.

Rationalizing the consumption of antibiotics is essential to preserve their effectiveness and ensure their availability for future generations.

We are going to achieve that through our Application and Website System that will Achieve, which involves optimizing the use of antibiotics through appropriate prescribing practices, surveillance and monitoring, and the development of alternative treatments.

Acknowledgement

We express our deep gratitude and appreciation to those who agreed to participate in this project.

First, we have to thank **Prof. Nabila Mohamed Hassan** faculty Dean for facilitating the educational process and making sure for providing all necessary tangible and mental basics needed for our success.

Special thanks go to **Dr. Eng. Samy Elsayed El Mokadem** ,our supervisor in this project for supporting us throughout the year. He Was very patient toward us and acted like a real mentor when we needed him. We benefited a lot from his knowledge. And **Dr. Abdelrahman El-Akhdar** for his support, constant supervision, and patience, as well as **Eng. Ahmed Rashad** for providing necessary information regarding the project.

We highly indebted to board members of **Faculty Of Pharmacy**, We Would Thank **Dr. Samir Othman** vice Dean Of the College Of Pharmacy And **Dr. Ahmed Essam** for providing this opportunity for us by trusting in us and provide the project with information and complete details.

Finally, we want to thank our family members for their support and encouragement and our friends for great and helpful suggestions.

Table of Contents

Abstract	I
Acknowledgment	II
List Of Abbreviations & Glossary	V
List Of Figures	VI
Chapter 1: Introduction	
1.1 Overview.....	1
1.2 Problem Definition	2
1.3 Objectives	3
1.4 Stack Holder	3
1.5 Work plan	4
Chapter 2: System Analysis	
2.1 Overview.....	5
2.2 Requirements	5
2.2.1 Functional requirements	5
2.2.2 Non- Functional requirements	6
2.3 System analysis diagrams	7
2.3.1 Use Case Diagram	10
2.3.2 Data Flow Diagram	11
2.3.3 Activity Diagram	18
2.3.4 Sequence Diagram	22
Chapter 3: System Design	
3.1 Overview	23
3.2 Entity Relationship Diagram	25
3.3 Schema Model	26
Chapter 4: System Implementation	
4.1 Over View	27
4.2 Developing tools	28
4.3 System Screen Shoots.....	32
4.4 Mobile Application Screen Shoots	44

Chapter 5: Conclusion, Future Work

5.1 Conclusion	50
5.2 Future Work	50
Appendices.....	51
References	76

List of Abbreviations & Glossary

- **ERD** Entity Relationship Diagram
- **CSS** Cascading Style Sheets
- **MVC** Model View Controller
- **Html** Hyper Text Markup Language
- **JS** JavaScript
- **API** Application Programming Interface
- **DBMS** Data Base Management System
- **UML** Unified Modeling Language
- **SEO** Search engine optimization

Glossary

Admin	Someone with full accessibility (permissions) who can •Add users to system. •Track Users Activities. •View the consumption statistics of antibiotic.
User	Someone with minimum accessibility on the system, the role of default user is only inserting and updating patient's records, and his permissions can be extended as needed.
Consumption Statistics	Admin can select a specific time (exp. Weekly or monthly) where he can see a full statistics data of the antibiotic consumed during that period

Figures List

Figure (1.1) Process Model.....	4
Figure (2.1) Use Case Diagram	10
Figure (2.2) Data Flow Diagram Context level.....	13
Figure (2.3) Data Flow Diagram Level 0.....	14
Figure (2.4) Activity Diagram	18
Figure (2.5) Sequence Diagram	22
Figure (2.6) Sequence Diagram.....	23
Figure (3.1) Entity Relationship diagram.....	25
Figure (3.2) Schema Model.....	26
Screen Shoots Of System.....	32
Screen Shoots Of Mobile Application	44

Deleted

Chapter 1

Introduction

1.1 Overview

Bacteria is a very serious disease, which may cause death sometimes, and treatment of bacteria using inappropriate antibiotics causes immunodeficiency, so we aim to create a system that aims to rationalize the consumption of antibiotics, by knowing the type of bacteria in the patient, by taking a sample from The patient's body, analysis, knowing the type of bacteria, and giving him the antibiotic that is resistant to this bacteria, and through that, the patient will take the appropriate antibiotic for him, because there are types of antibiotics that treat more than one type of bacteria, so through the correct diagnosis of the patient and knowing the bacteria that he has, he will take the antibiotic that is resistant to this bacteria, This will save money for the hospital and the patient due to the high price of antibiotics.

1.2 Problem

Each antibiotic can be used to treat more than one type of bacteria, and the doses of antibiotics required to treat a patient vary. These doses vary according to the patient's age, gender, health condition and other factors. Therefore, it is necessary to have a system followed in the consumption of antibiotics that aims to rationalize the consumption of antibiotics in their places of distribution, whether hospitals, clinics or pharmacies.

1.3 Project Objectives:

The project aims to build a system that helps rationalize the consumption of antibiotics. The best way to rationalize consumption is to monitor the number of patients, the types of diseases they have, and the doses required for the treatment process.

1.3.1 Objectives

- Detecting if an antibiotic is being consumed inappropriately
- Setting appropriate dosage instructions for each patient, according to the case, during the patient registration process.
- Monitoring and extracting patients' statistics at the hospital
- Monitoring and extracting statistics of antibiotic consumption in the hospital.
- Conserve time and effort.
- Injecting new technology in O6U services.
- Reduce the number of consumed antibiotics.

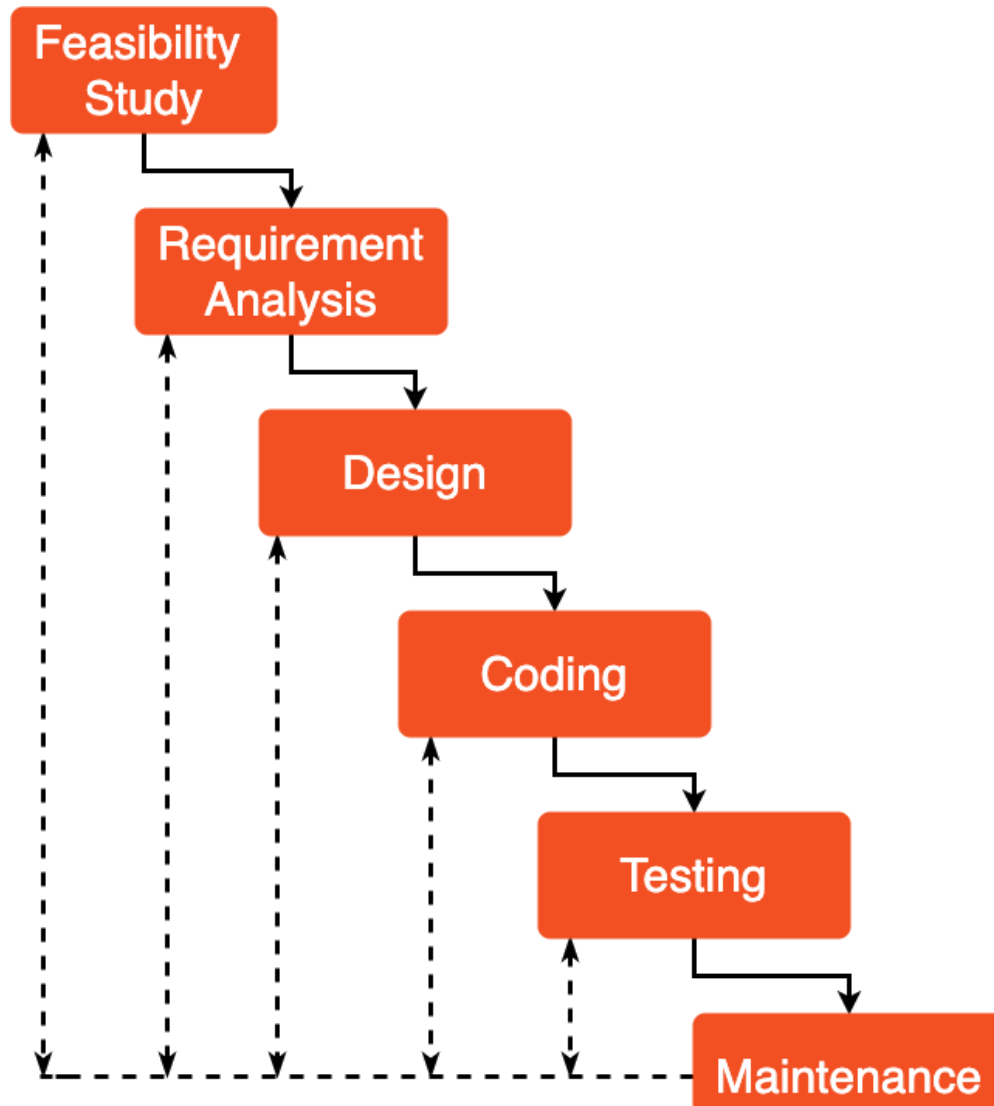
1.4 Stakeholders

- Faculty Of Pharmacy.
- Hospitals.

1.5 Work Plan

It's necessary to use waterfall process model, since all requirements of the system are gathered and specified, and it gives us a space to clearly define each stage perfectly.

Process Model



Chapter 2

System Analysis

Deleted

2.1 Overview

This chapter include system analysis and design. It describes how the system works through discuss functional and non-functional requirements, Context diagram, Data Flow Diagram (DFD), Entity Relationship Diagram (ERD), etc.

2.2 Requirements:

This section discusses how the main functions work and includes its analysis to make the system clearer for reader and understand what happen.

2.2.1 functional Requirements:

A functional requirement (FR) document defines the functionality of a system or one of its subsystems. It also depends upon the type of software, expected users and the type of system where the software is used.

- Ability to register patients' data with the type of antibiotic consumed.
- Ability to manage patients' records.
- Ability to manage antibiotic quantity in hospital.
- Ability to manage antibiotic quantity in hospital.
- Ability to add users with different roles on the system.
- Ability to manage users and see their activities.

2.2.2 Non-functional Requirements:

A Non-Functional requirement (NFR) is a requirement that specifies criteria that can be used to judge the operation of the system, rather than specific behaviors. Non-functional requirements are often called “quality attributes” of a system. Nonfunctional requirements

such as:

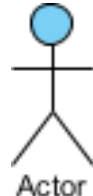
- Data integrity
- System security
- System reliability
- System dependability

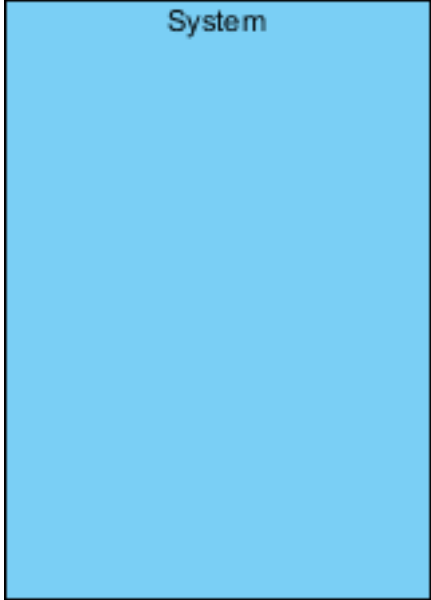
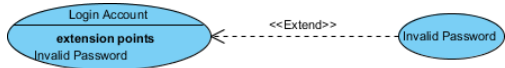
2.3 System analysis diagrams:

2.3.1 Use Case :

A UML use case diagram is the primary form of system/software requirements for a new software program underdeveloped. Use cases specify the expected behavior (what), and not the exact method of making it happen (how).

Components of Use case Diagram:

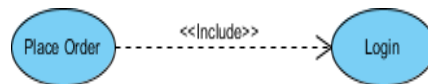
Actor - Someone interacts with use case (system function). - Actor plays a role in the business. - Similar to the concept of user, but a user can play different roles. - Actor has a responsibility toward the system (inputs), and Actor has expectations from the system (outputs). - Actor triggers use case(s).	
Use case - System function (process – automated or manual) - Each Actor must be linked to a use case, while some use cases may not be linked to actors.	
Communication Link - The participation of an actor in a use case is shown by connecting an actor to a use case by a solid link. - Actors may be connected to use cases by associations,	

indicating that the actor and the use case communicate with one another using messages.	
Boundary of system • The system boundary is potentially the entire system as defined in the requirements document. • For large and complex systems, each module may be the system boundary. • For example, for an ERP system for an organization, each of the modules such as personnel, payroll, accounting, etc. • can form a system boundary for use cases specific to each of these business functions. • The entire system can span all of these modules depicting the overall system boundary	
Use case Relationships Extends • Indicates that an “Invalid Password” use case may include (subject to specified in the extension) the behavior specified by base use case “Login Account”. • Depict with a directed arrow having a dotted line. The tip of arrowhead points to the base use case and the child use case is connected at the base of the arrow.	

- The stereotype “<<extends>>” identifies as an extend relationship

Include

- When a use case is depicted as using the functionality of another use case, the relationship between the use cases is named as include or uses relationship.
- A use case includes the functionality described in another use case as a part of its business process flow.
- An include relationship is depicted with a directed arrow having a dotted line. The tip of arrowhead points to the child use case and the parent use case connected at the base of the arrow.
- The stereotype “<<include>>” identifies the relationship as an include relationship.



Purpose of Use Case Diagram is

- The ability to save patient records (name, age, gender, blood sample, antibiotic, antibiotic portion)
- The ability to modify patient records
- The ability to view the statistics of antibiotic consumption
- The ability to track Users activity

Use case:

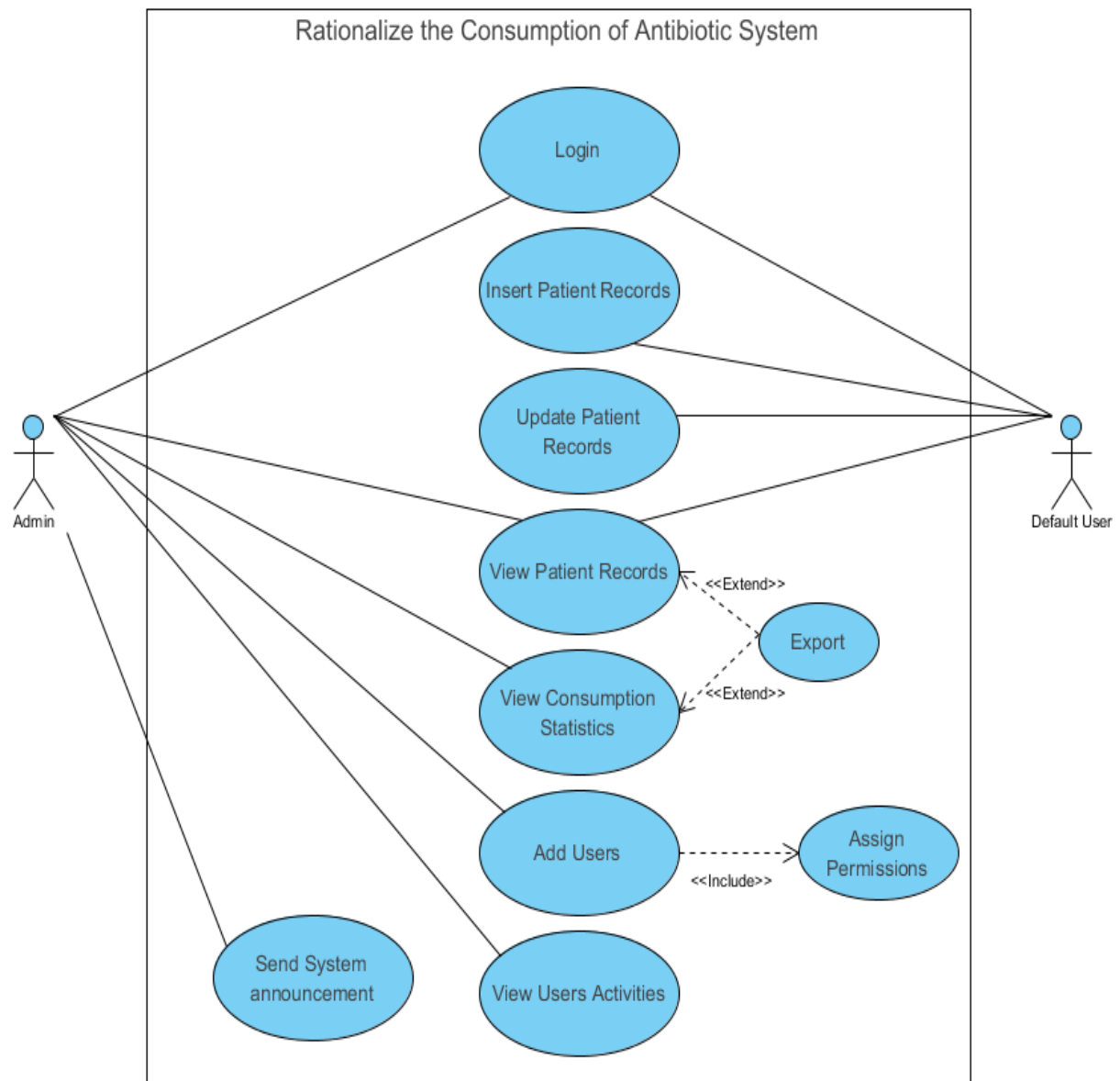


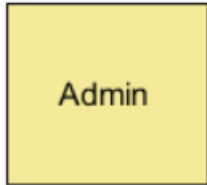
Figure (2.1) Use Case Diagram

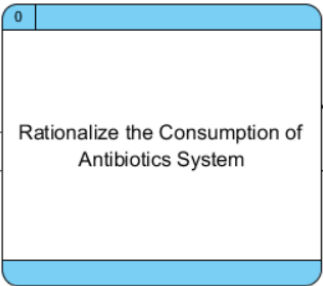
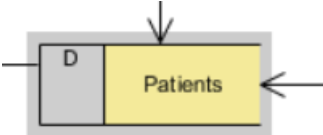
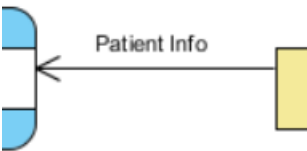
2.3.2 Dataflow Diagram :

It shows how information enters and leaves the system, what changes the information and where information is stored. The purpose of a DFD is to show the scope and boundaries of a system as a whole. It may be used as a communications tool between a systems analyst and any person who plays a part in the system that acts as the starting point for redesigning a system.

It is usually beginning with a context diagram as level 0 of the DFD diagram, a simple representation of the whole system. To elaborate further from that, we drill down to a level 1 diagram with lower-level functions decomposed from the major functions of the system. This could continue to evolve to become a level 2 diagram when further analysis is required. Progression to levels 3, 4 and so on is possible but anything beyond level 3 is not very common. Please bear in mind that the level of detail for decomposing a particular function depending on the complexity that function.

DFD Components

External Entity An external entity can represent a human, system or subsystem. It is where certain data comes from or goes to. It is external to the system we study, in terms of the business process. For this reason, people used to draw external entities on the edge of a diagram.	
Process	

A process is a business activity or function where the manipulation and transformation of data take place. A process can be decomposed to a finer level of details, for representing how data is being processed within the process.	
Data Store A data store represents the storage of persistent data required and/or produced by the process.	
Data Flow A data flow represents the flow of information, with its direction represented by an arrowhead that shows at the end(s) of flow connector.	

Dataflow of Rationalize the Consumption of Antibiotic System

Context Level

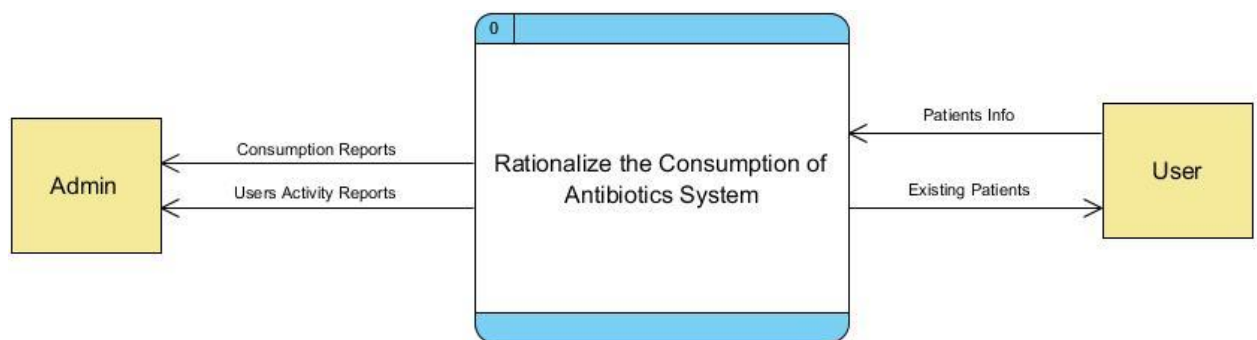


Figure (2.2) Data Flow Diagram Context level

DFD Level 1

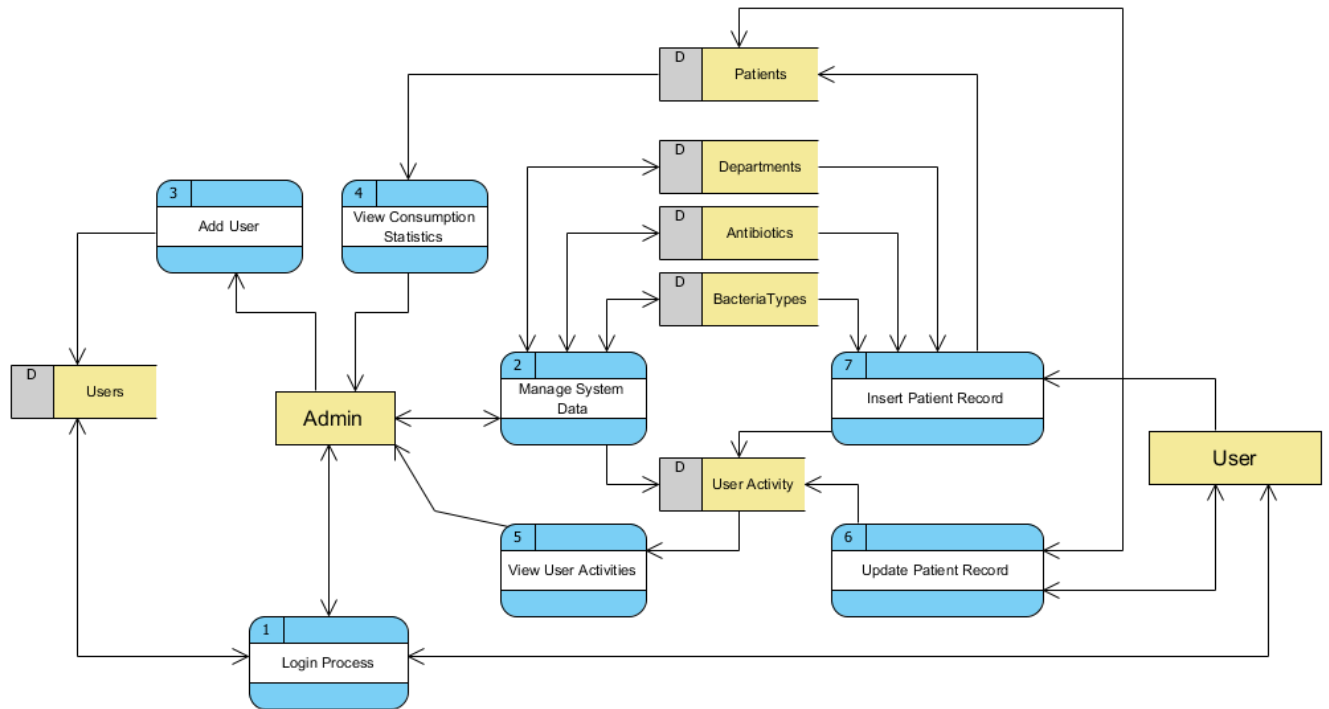
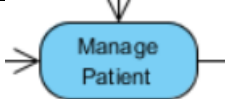


Figure (2.3) Data Flow Diagram Level 0

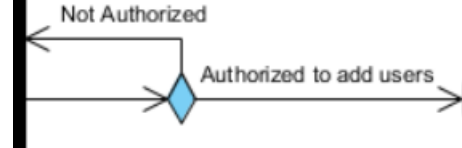
2.3.3 Activity Diagram

Activity Diagrams describe how activities are coordinated to provide a service which can be at different levels of abstraction. Typically, an event needs to be achieved by some operations, particularly where the operation is intended to achieve a number of different things that require coordination, or how the events in a single use case relate to one another, in particular, use cases where activities may overlap and require coordination.

Notations of Activity Diagram

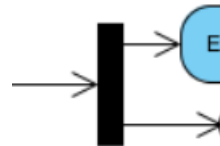
Action : A task to be performed.	
Initial Node : Portrays the beginning of a set of actions or activities.	
Activity Final Node : Stop all control flows and object flows in an activity (or action)	
Decision Node :	

Represent a test condition to ensure that the control flow or object flow only goes down one path.



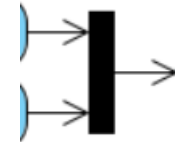
Fork Node :

Split behavior into a set of parallel or concurrent flows of activities (or actions).



Join Node :

Bring back together a set of parallel or concurrent flows of activities (or actions).



The Next Activity Diagram Describes:

- Login Activity
- Insert new patient record activity
- Update existing patient record activity
- View/Export Antibiotic Consumption Statistics
- Add new user activity
- Manage System settings (Departments, Antibiotic Types, Bacteria Types)

Activity Diagram of Rationalize the consumption of Antibiotics System

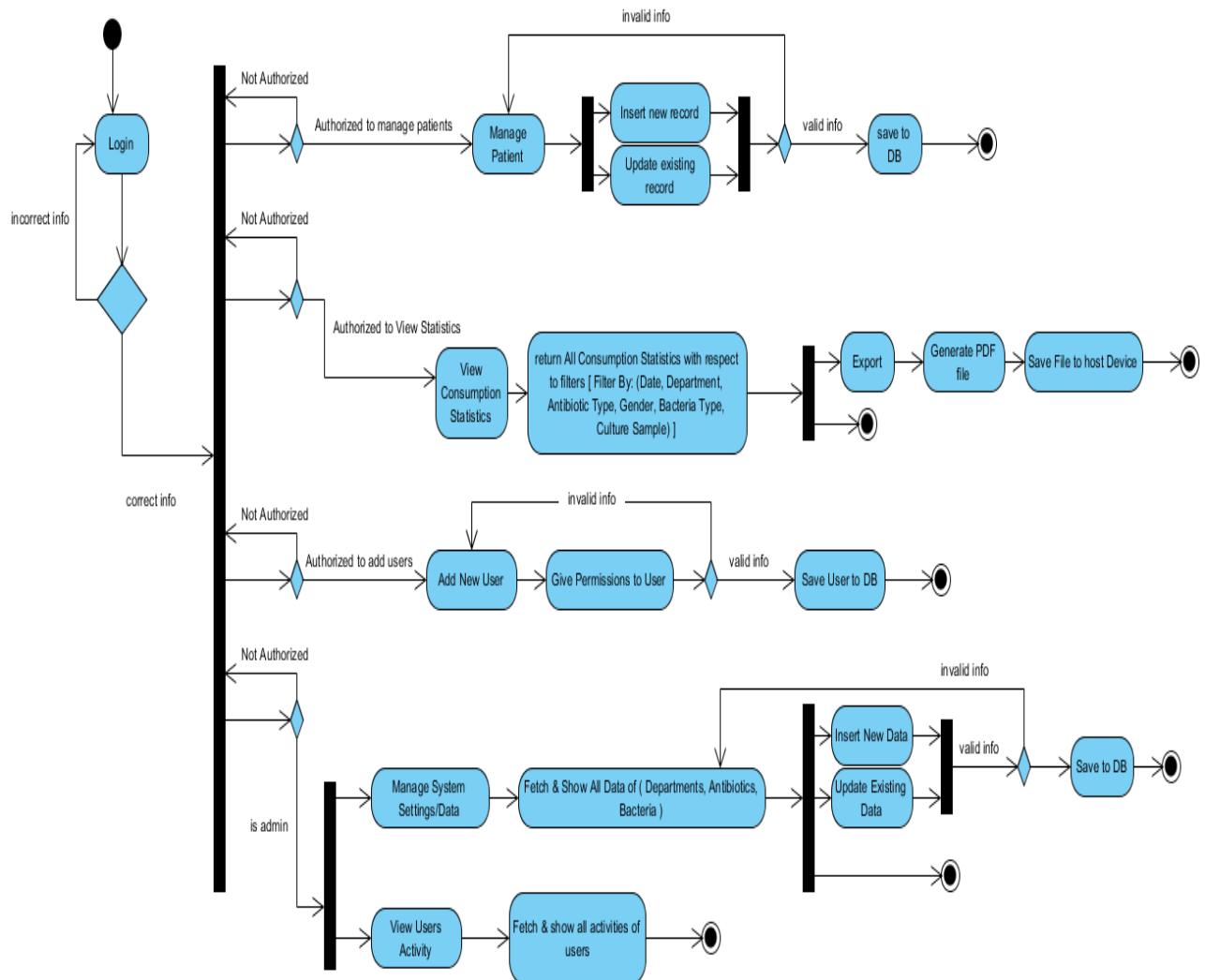


Figure (2.4) Activity Diagram

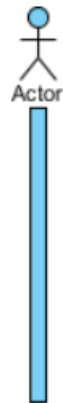
2.3.4 Sequence Diagram

Sequence Diagrams are interaction diagrams that detail how operations are carried out. They capture the interaction between objects in the context of a collaboration. Sequence Diagrams are time focus and they show the order of the interaction visually by using the vertical axis of the diagram to represent time what messages are sent and when.

Components Of Sequence Diagram :

Actor:

- a type of role played by an entity that interacts with the subject
- external to the subject (i.e., in the sense that an instance of an actor is not a part of the instance of its corresponding subject).
- represent roles played by human users, external hardware, or other subjects.

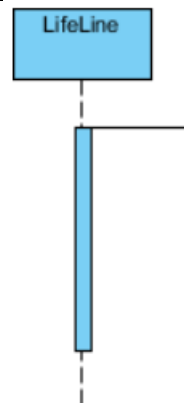


Lifeline :

- A lifeline represents an individual participant in the Interaction.

**Activations :**

- A thin rectangle on a lifeline) represents the period during which an element is performing an operation.
- The top and the bottom of the of the rectangle are aligned with the initiation and the completion time respectively

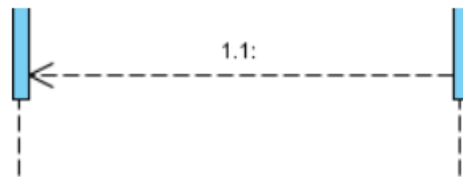
**Call Message :**

- Call message is a kind of message that represents an invocation of operation of target lifeline.



Return Message :

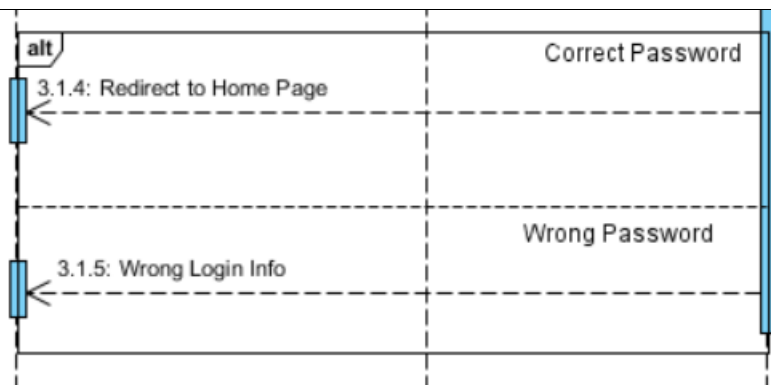
- Return message is a kind of message that represents the pass of information back to the caller of a corresponded former message.

**Self-Message :**

- Self-message is a kind of message that represents the invocation of message of the same lifeline.

**Alt Fragment :**

- Alternative multiple fragments: only the one whose condition is true will execute.



Sequence Diagram of Inserting new Patient Record

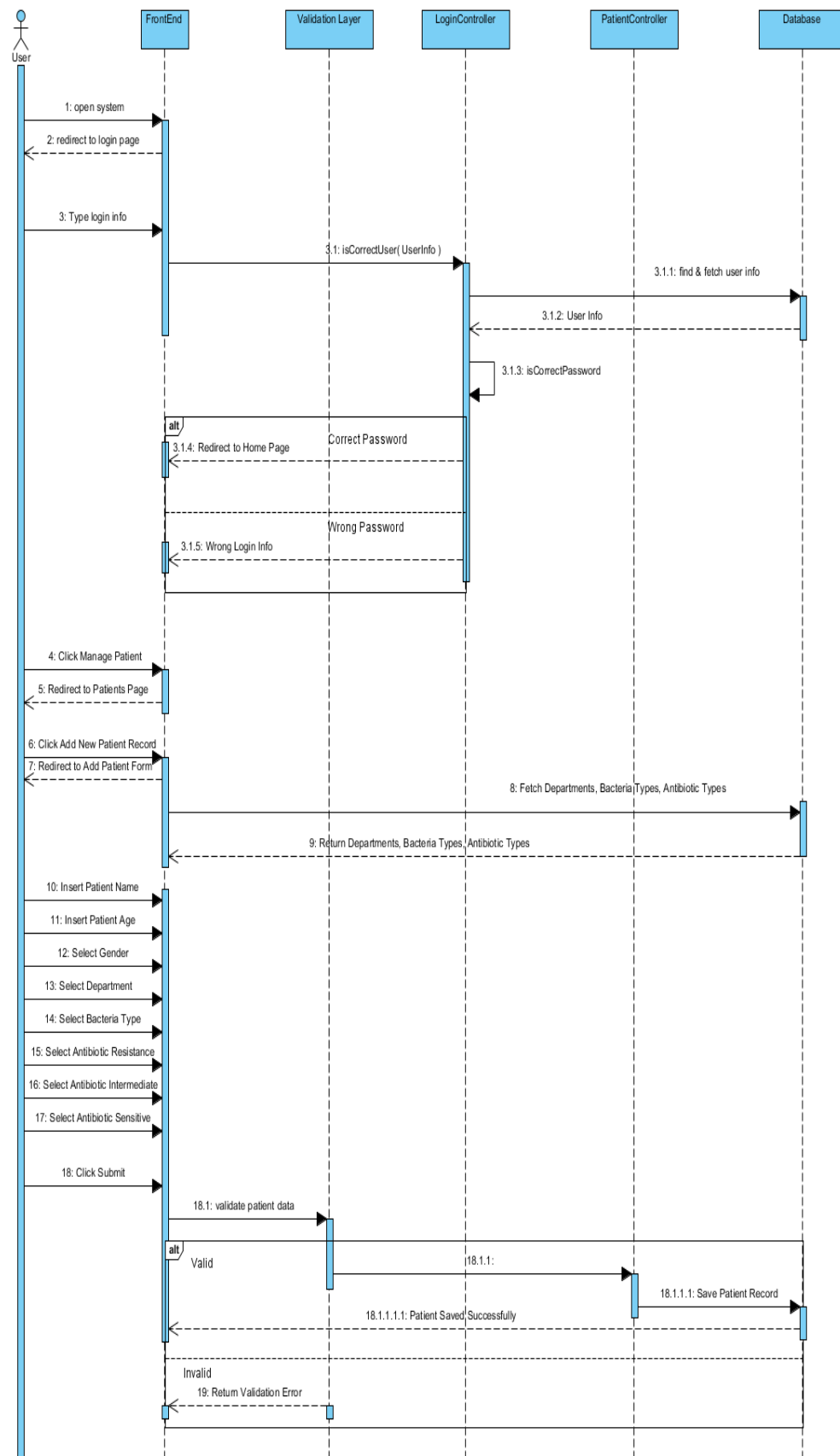


Figure (2.5) Sequence Diagram 1.0

Sequence Diagram of Viewing Antibiotics Consumption Statistics

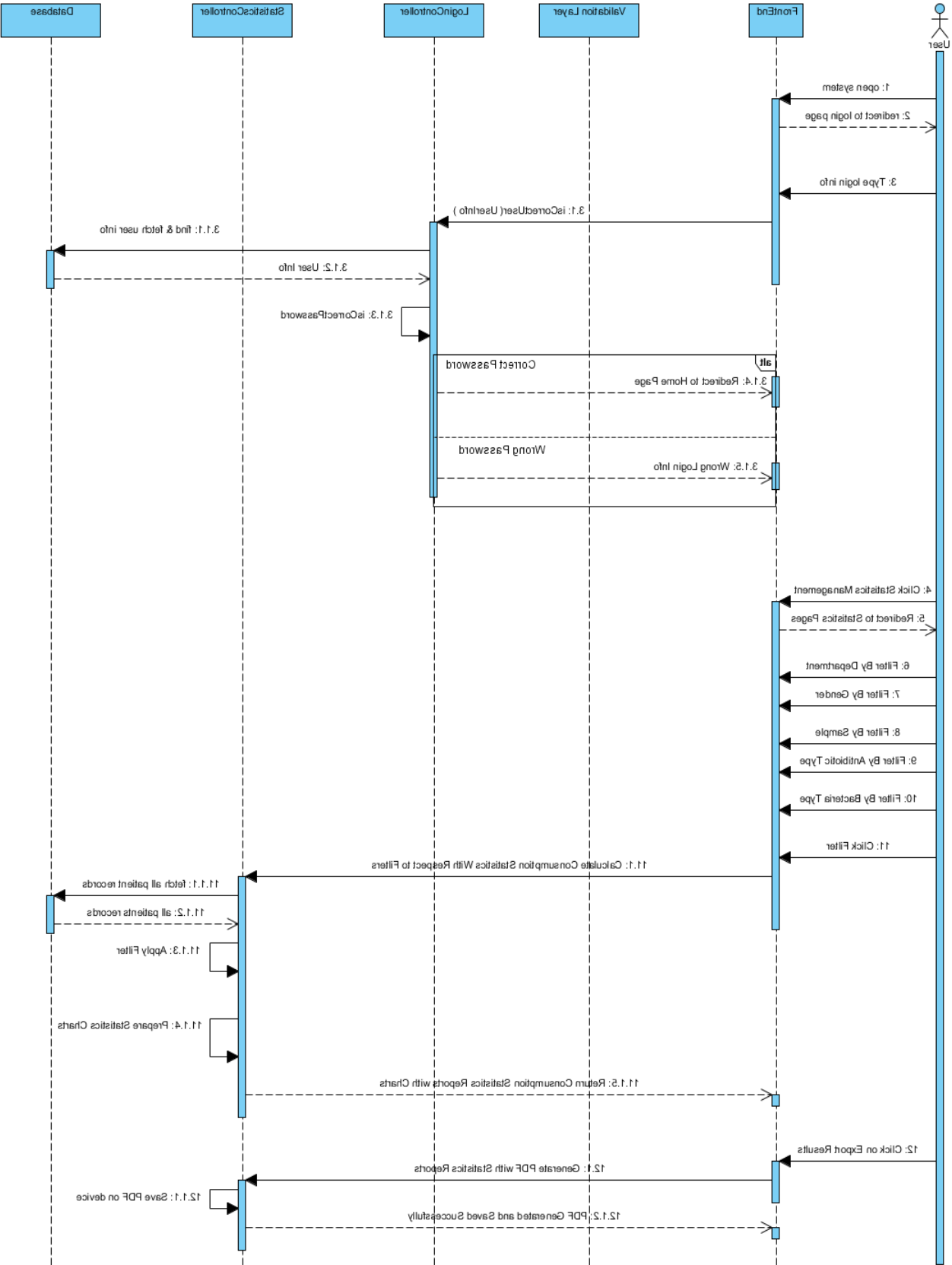


Figure (2.6) Sequence Diagram 1.1

Chapter 3

System Design

3.1 Overview

The process of defining the architecture, product design, modules, interfaces, and data for a system in order to meet specific criteria is known as systems design. The application of systems theory to product development is known as systems design. The disciplines of systems analysis, systems architecture, and systems engineering have some overlap.

3.2 Entity Relationship Diagram

The relationships between entity sets recorded in a database are depicted In an ERD An object, or a data component, is an entity in this context collection of a similar is referred to as an entity set These entities can have values defined via attributes, an ERD depicts the logical flow by defining the entities , their attributes, and the relationships between them It explains interconnected objects of interest, entity types (which classify the things of interest) and relationships (which might exist between entities) make up the foundation of an ER model.

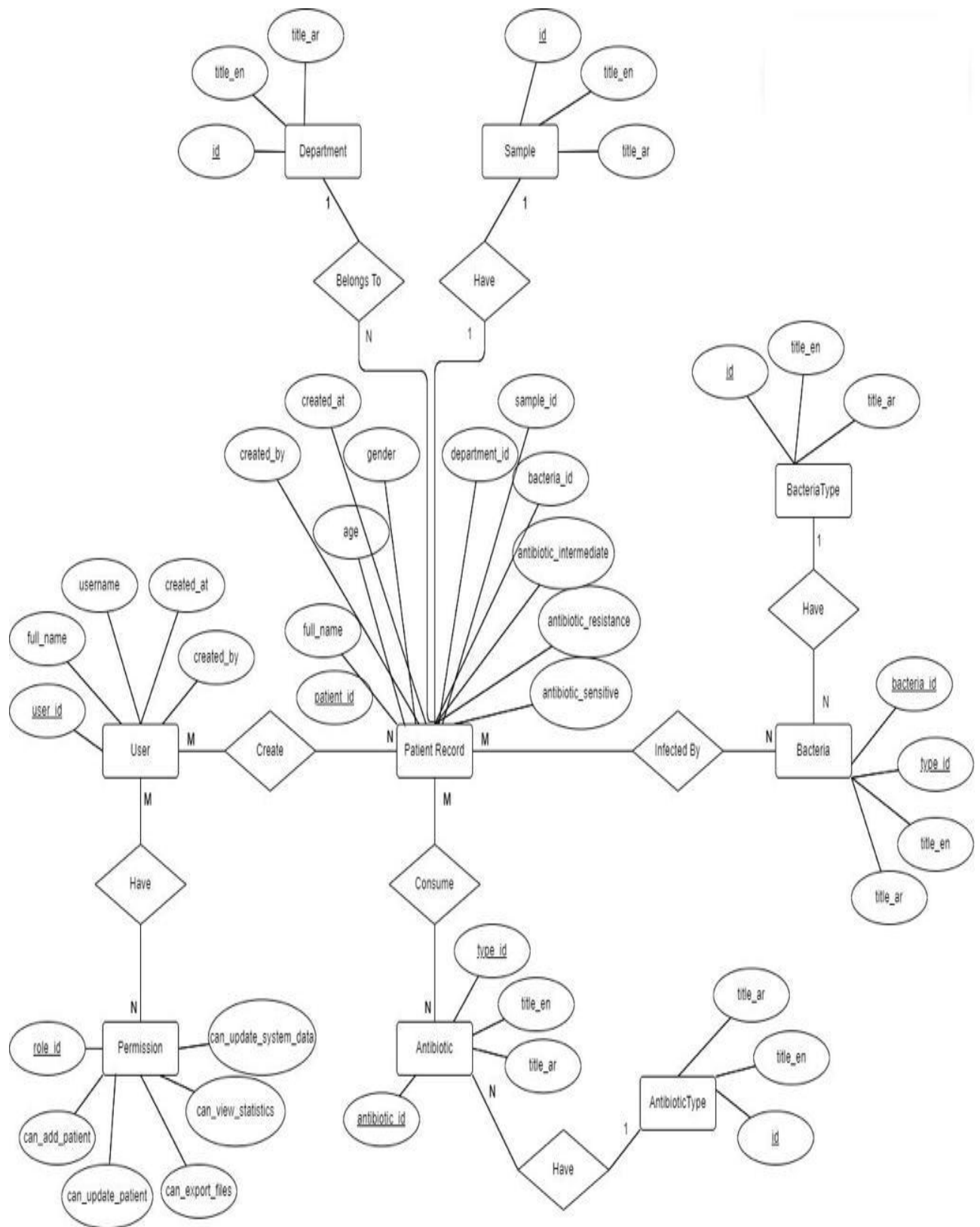


Figure (3.1) Entity Relationship Diagram

3.3 Schema Model

is a form of data model that establishes a database's logical structure, it dictates show data may be stored, structured, and manipulated in the first place, the relational model, which uses a table-based format, is the most used database model.

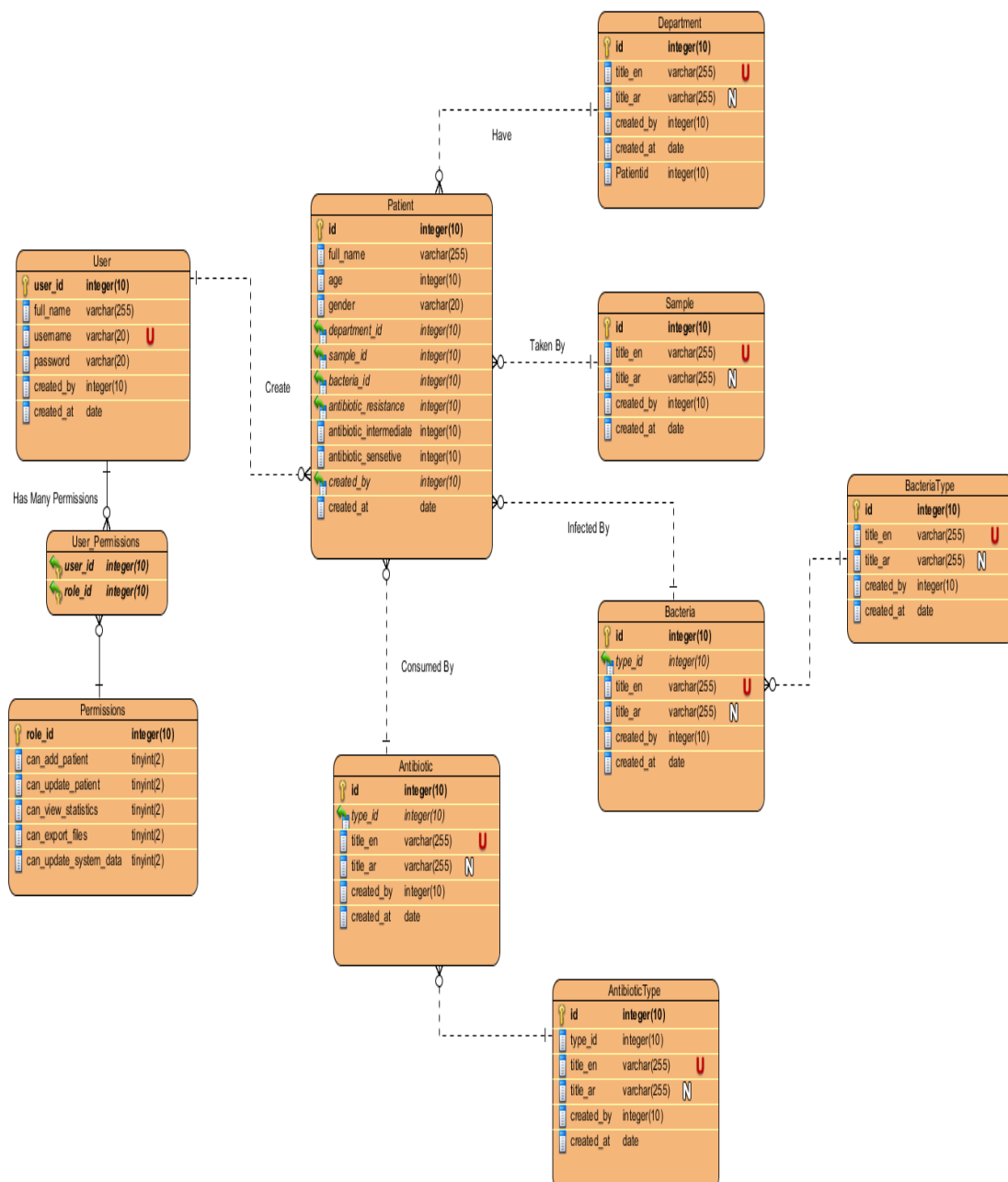


Figure (3.2) Database Model

Deleted

Chapter 4

System Implementation

4.1 Overview:

This chapter will provide more detailed information about the technologies used in the project and other important information about the vision to be clear.

4.2 Tools

With the help of high-end tools we gathered and combined we had the opportunity to deliver and accomplish all the project objectives in the easiest and most efficient way.

In building our website we used the following tools:

Front-end :

- html5
- css3
- JavaScript
- jQuery
- bootstrap

Back-end :

- Php
- Laravel
- My SQL
- MVC

Mobile Application:

- Flutter

4.2.1 web development:

Technology is an important aspect of our daily life. It allows us to carry out everyday chores but also making room for extraordinary ideas. During the web development process, web developers create web pages and applications for the Internet or a private network. In general, web development refers to the tasks involved in creating websites for intranet or internet hosting.

Web design, web content development, client side/server-side scripting, and network security settings are all part of the web development process. Web development, in a wide sense, refers to all of the actions, updates, and operations required to create, maintain, and administer a website in order to assure its optimal functionality, user experience, and speed.

It may also include, but is not required to, all of the strategic measures required to assure its proper ranking in search engine results. Those jobs usually fall under a different category, notably search engine optimization (SEO) tasks.

4.2.2 Front-end development:

Front-end web development, also known as client-side development, is the process of building code for a website or web application in Hyper Text Markup Language (HTML), Cascading Style Sheets (CSS), and JavaScript so that users can see and interact with it directly.

It's important to understand the fundamental differences between front-end development, back-end development, and full stack development before diving deeper into this topic. Simply described, a front-end developer is a programmer that works on the components of an app or website that consumers view and interact with.

A back-end developer, on the other hand, is responsible for the 5 developer, on the other hand, is someone who works on the entire design process from beginning to end. Front-end development is required for various websites and web apps. Consider a single-page information website. Back-end development, on the other hand, is usually required for anything that is functional. So front-end development is responsible for everything you see on the page – and by that we mean anything that

happens on the user's computer. Back-end development encompasses anything that receives data from a server.

When an app is in use, requests are sent from the frontend (the page) to the back end (i.e. server), which subsequently sends information in plain format.

The front-end styles it before rendering it, whereas the backend is server-side programming that communicates with the database

4.2.3 Goals and challenges in front-end development:

Accessibility: This is where they ensure that everyone can easily use the website or app, regardless of their mobile device (think smartphones and tablets).

Front-end developers use this method to ensure that their site displays appropriately in all browsers and on all devices. This is accomplished utilizing CSS stylesheets and a responsive web design. •

Render time: Here, performance is measured in terms of render time. As a result, front-end developers work on the code to ensure that the site/app loads quickly.

Rapid Development: This refers to a development method that is both swift and efficient.

4.2.4 Database:

A database is an organized collection of data, generally stored and accessed electronically from a computer system. Where databases are more complex they are often developed using formal design and modeling techniques.

The database management system (DBMS) is the software that interacts with end users, applications, and the database itself to capture and analyze the data.

The DBMS software additionally encompasses the core facilities provided to administer the database. The sum total of the database, the DBMS and the associated applications can be referred to as a “database system”. Often the term “database” is also used to loosely refer to any of the DBMS, the database system or an application associated with the database.

Computer scientists may classify database-management systems according to the database models that they support. Relational databases became dominant in the 1980s.

These model data as rows and columns in a series of tables, and the vast majority use My SQL for writing and querying data. In the 2000s, non-relational databases became popular, referred to as NoSQL because they use different query languages.

4..2.5 Backend Development:

What is an API?

API is the acronym for Application Programming Interface, which is a software intermediary that allows two applications to talk to each other. Each time you use an app like Facebook, send an instant message, or check the weather on your phone, you're using an API.

What is Python?

Python is an interpreted high-level general-purpose programming language. Its design philosophy emphasizes code readability with its use of significant indentation. Its language constructs as well as its object-oriented approach aim to help programmers write clear, logical code for small and large-scale projects

4..2.6 Mobile Application Technology

Flutter

Flutter is a mobile app development framework created by Google. It allows developers to create high-performance, visually appealing, and natively compiled applications for Android, iOS, web, and desktop platforms using a single codebase. Flutter uses Dart, a modern programming language that is easy to learn and offers features such as hot reload, which allows developers to see changes in real-time as they make modifications to the code. Flutter also provides a rich set of pre-built widgets and tools that make it easy to create complex UI designs and animations. Due to its simplicity, flexibility, and performance, Flutter has gained popularity among developers, and it is widely used to develop mobile apps for various industries, including healthcare, finance, e-commerce, and gaming.

4.3 System Screen Shoots:

RCA
System

 mido@rca.com



☐ Keep me logged in

Log in



Login Page On Website

Admin **Super Admin**

 Dashboard

 Statistics

 Patients

 Antibiotics

 Bacterias

 Departments

 Cultures

 Users

 Log Out

Overview



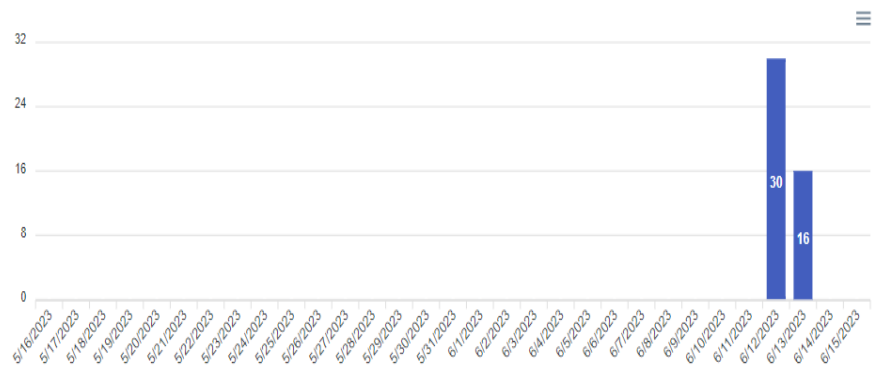
User Total
2



Patient Records
46

Patients

Shows number of patients added to system each day for the last 31 days



Dashboard Page

(Show Number Of Users Who Use System , And Number Of Patients Recorded In The System)

Admin Super Admin

Dashboard

Statistics

Patients

Antibiotics

Bacterias

Departments

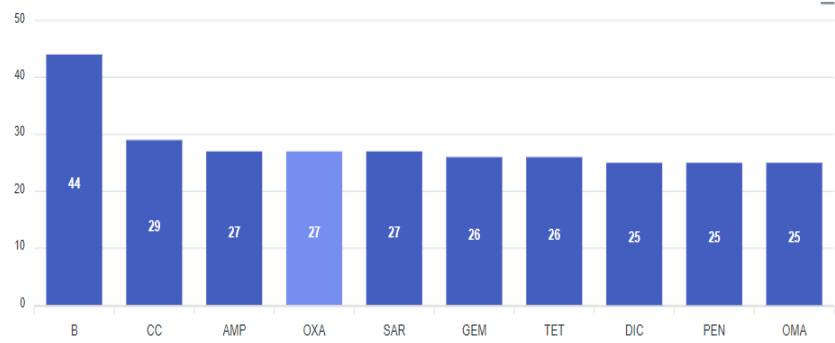
Cultures

Users

Log Out

Antibiotic Usage

Shows top 10 consumed antibiotic



2023 © O6U University

Show Top 10 Antibiotic Consumed

Admin Super-Admin				
Dashboard				
Statistics				
Patients				
Antibiotics				
Bacterias				
Bacterias				
Bacteria Groups				
Departments				
Cultures				
Users				

Bacteria Name	Antibiotic Statistics			
	Antibiotic	%R	%I	%S
	Amoxicillin and Clavulanate	100	0	0
	Ampicillin	100	0	0
	Penicillin V Potassium	100	0	0
	Cefaclor	100	0	0
	Cefadroxil	100	0	0
	Cephalexin	100	0	0
	Cefiderocol	100	0	0
	Ceftriaxone	100	0	0
	Cefuroxime	100	0	0
	Tygacil	50	0	50
	Ciprofloxacin	100	0	0
	Levofloxacin	100	0	0
	Gemifloxacin	100	0	0
	Doxycycline	100	0	0
Acinetobacter (XDR)	Meropenem	100	0	0

Show Bacteria And Effectiveness Of Antibiotics With It

Mido Said Admin				
Dashboard				
Statistics				
Patients				
Users				
Log Out				

Bacteria Statistics

Filter

Select Date From

Select Date To

Department

-- Select Department --

Gender

-- Select Gender --

Bacteria

Select Bacteria

Filter

Cancel

Export

Bacteria Name	Antibiotic Statistics			
	Antibiotic	%R	%I	%S
	Amoxicillin and Clavulanate	100	0	0

Filter Process If I Want To Search About Specific Bacteria Or Specific Period

Bacteria Name	Antibiotic Statistics			
Acinetobacter (XDR)	Antibiotic	%R	%I	%S
	Amoxicillin and Clavulanate	100	0	0
	Ampicillin	100	0	0
	Penicillin V Potassium	100	0	0
	Cefaclor	100	0	0
	Cefadroxil	100	0	0
	Cephalexin	100	0	0
	Cefiderocol	100	0	0
	Ceftriaxone	100	0	0
	Cefuroxime	100	0	0
	Tygacil	50	0	50
	Ciprofloxacin	100	0	0
	Levofloxacin	100	0	0
	Gemifloxacin	100	0	0
	Doxycycline	100	0	0
	Meropenem	100	0	0
	Ertapenem	100	0	0
	Polymixin B	33.33	0	66.67
	Amoxicillin	100	0	0
	Dicloxacillin	100	0	0
	Oxacillin	100	0	0
	Demeclocycline	100	0	0
	Doxycycline	100	0	0
	Eravacycline	100	0	0
	Minocycline	100	0	0
	Omadaacycline	100	0	0
	Sarecycline	100	0	0
	Tetracycline	100	0	0
	Delafloxacin	100	0	0
	Moxifloxacin	100	0	0

Also Can Export Result In Pdf Form Like This

Admin **Super Admin**

Dashboard
Statistics
Patients
Antibiotics
Antibiotics
Antibiotic Groups
Bacterias
Bacterias
Bacteria Groups
Departments

Create Patient

Name

Age

Gender

Department

Culture

Bacteria

Resistance Antibiotics

Intemeditate Antibiotics

Sensitive Antibiotics

Process Of Adding New Patient

Admin **Super Admin**

Dashboard
Statistics
Patients
Antibiotics
Bacterias
Departments
Cultures
Users
Log Out

Patients

Search...

Search Create New Patient

Name	Age	Gender	Department	Culture	Bacteria	Created By	Actions
سماء محمد سليمان	73	female	رعاية مركزية ICU	Sputum Culture and Sensitivity	Klebsiella spp(XDR)	Admin	Edit Delete
مريم محمد محمود	74	female	رعاية مركزية ICU	Blood C & S Aerobic-Anerobic	Staphylococcus aureus (MRSA)	Admin	Edit Delete
حمدي عوده احمد	75	male	رعاية مركزية ICU	Sputum Culture and Sensitivity	Klebsiella spp MDR	Admin	Edit Delete
مصطفى محمود	55	male	رعاية مركزية ICU	Blood C & S Aerobic-Anerobic	Klebsiella spp(XDR)	Admin	Edit Delete

As Son As I Created Patient , Patient Record Will be Added To Patients Records , Also I Can Edit Patient Data And I Can Search About Specific Patient From Search Bar.

Admin Super Admin

Dashboard
Statistics
Patients
Antibiotics
Antibiotics
Antibiotic Groups
Bacterias
Departments
Cultures
Users

Antibiotics

Add Antibiotic

ID	Abbreviation	Name	Group	Patients	Created By	Actions
1	AMX	Amoxicillin	Penicillins	23	Admin	Edit
2	AMC	Amoxicillin and Clavulanate	Penicillins	24	Admin	Edit
3	AMP	Ampicillin	Penicillins	27	Admin	Edit
4	DIC	Dicloxacillin	Penicillins	25	Admin	Edit
5	OXA	Oxacillin	Penicillins	27	Admin	Edit
6	PEN	Penicillin V Potassium	Penicillins	25	Admin	Edit
7	DMC	Demeclocycline	Tetracyclines	24	Admin	Edit
8	DOX	Doxycycline	Tetracyclines	22	Admin	Edit
9	ERA	Eravacycline	Tetracyclines	18	Admin	Edit
10	MIN	Minocycline	Tetracyclines	20	Admin	Edit

This Page Contain All Antibiotics Recorded In The System ,I Can Add New Antibiotic And I Can Update Data Of Antibiotic.

Admin Super Admin

Dashboard
Statistics
Patients
Antibiotics
Antibiotics
Antibiotic Groups
Bacterias
Departments
Cultures
Users

Antibiotic Groups

Add Group

ID	Name	Number of Antibiotics	Created By	Actions
1	Penicillins	7	Admin	Edit
2	Tetracyclines	7	Admin	Edit
3	Cephalosporins	14	Admin	Edit
4	Fluoroquinolones	6	Admin	Edit
5	Lincomycins	4	Admin	Edit
6	Macrolides	7	Admin	Edit
7	Sulfonamides	3	Admin	Edit
8	Glycopeptide Antibiotics	6	Admin	Edit
9	Aminoglycosides	3	Admin	Edit
10	Carbapenems	5	Admin	Edit

This Page Contain Groups Of Antibiotics I Can Also Add New Group ,And I Can Update Data Of Antibiotics Groups.

Admin	Super Admin
Dashboard	
Statistics	
Patients	
Antibiotics	
Bacterias	
Bacterias	
Bacteria Groups	
Departments	
Cultures	
Users	

Bacterias					
Add Bacteria					
ID	Name	Group	Patients	Created By	Actions
1	Staphylococcus aureus	Gram-positive cocci	0	Admin	Edit Delete
2	Staphylococcus epidermidis	Gram-positive cocci	0	Admin	Edit Delete
3	Coagulase-negative staphylococci, not specified	Gram-positive cocci	0	Admin	Edit Delete
4	Other coagulase-negative staphylococci (CNS)	Gram-positive cocci	0	Admin	Edit Delete
5	Staphylococcus spp., not specified	Gram-positive cocci	0	Admin	Edit Delete
6	Streptococcus pneumoniae	Gram-positive cocci	0	Admin	Edit Delete
7	Streptococcus agalactiae	Gram-positive cocci	0	Admin	Edit Delete
8	Streptococcus pyogenes	Gram-positive cocci	0	Admin	Edit Delete
9	Other haemol. streptococcae (C, G)	Gram-positive cocci	0	Admin	Edit Delete
10	Streptococcus spp., other	Gram-positive cocci	0	Admin	Edit Delete

This Page Show Bacteria Name And Group Followed To And Number Of Patients Who Infected By It, Also I Can Add New Bacteria.

Admin	Super Admin
Dashboard	
Statistics	
Patients	
Antibiotics	
Bacterias	
Bacterias	
Bacteria Groups	
Departments	
Cultures	
Users	

Bacteria Groups				
Add Group				
ID	Name	Number of Bacteria	Created By	Actions
1	Gram-positive cocci	20	Admin	Edit
2	Gram-negative cocci	15	Admin	Edit
3	Gram-positive bacilli	31	Admin	Edit
4	Enterobacteriaceae	29	Admin	Edit
5	Gram-neg., non-enterobacteriaceae	14	Admin	Edit
6	Anaerobic bacilli	2	Admin	Edit
7	Other bacteria	15	Admin	Edit
8	Fungi	12	Admin	Edit
9	Virus	0	Admin	Edit Delete

This Page Show Groups Of Bacteria, Also I Can Add New Group.

Admin **Super Admin**

 Dashboard

 Statistics ▼

 Patients

 Antibiotics ▼

 Bacterias ▼

 **Departments**

 Cultures

 Users ▼

 Log Out

Departments

Add Department

ID	Name	Number of Patients	Created By	Actions
1	Internal Medicine	1	Admin	Edit
2	General Surgery	0	Admin	Edit Delete
3	Heart Care	0	Admin	Edit Delete
4	Intensive Care	1	Admin	Edit
5	Open Heart	1	Admin	Edit
6	Orthopedic Surgery	1	Admin	Edit
7	رعاية مركزة ICU	40	Admin	Edit
8	قسم قلب CCU	2	Admin	Edit

This Page Show Departments In The Hospital In Which Patient Reserved , I Can Add New Department And Update Data Of Departments.

Admin **Super Admin**

 Dashboard

 Statistics ▼

 Patients

 Antibiotics ▼

 Bacterias ▼

 Departments

 Cultures

 Users ▼

 Log Out

Cultures

Add Culture

ID	Name	Number of Patients	Created By	Actions
1	Blood C & S Aerobic-Anerobic	12	Admin	Edit
2	Urine Culture and Sensitivity	17	Admin	Edit
3	Stool	0	Admin	Edit Delete
4	Sputum Culture and Sensitivity	10	Admin	Edit
5	Cerebrospinal fluid (CSF)	0	Admin	Edit Delete
6	Peritoneal fluid	0	Admin	Edit Delete
7	Pleural fluid	0	Admin	Edit Delete
8	Synovial fluid	0	Admin	Edit Delete
9	Other	0	Admin	Edit Delete

This Page Show Cultures Through Which I Know Which Type Of Bacteria Patient Have And Give Him Appropriate Antibiotic, Also I Can Add New Culture.

Admin **Super-Admin**

- Dashboard
- Statistics
- Patients
- Antibiotics
- Bacterias
- Departments
- Cultures
- Users**
- Log Out

Enter Your Name

Enter Your Email

Select Role Name

Choose Password

Choose Confirm Password

Add User

This Page Show The Process Of Adding New User To System ,And Select His Role On The System Which Data He Can Access And Which Not.

Admin **Super-Admin**

- Dashboard
- Statistics
- Patients
- Antibiotics
- Bacterias
- Departments
- Cultures
- Users**
- Log Out

User Management Control

Dashboard / User Mangement

For user to check they list

User Datatable

ID	Full Name	Email Address	Phone Number	Status	Role Name	Modify
1	Admin	admin@rca.com	01115785997	Active	Super Admin	 
2	Mido Said	mido@rca.com			Admin	  

2023 © O6U University

This Show If I Want To Modify User Data And Role Which He Have.

Admin

Super-Admin

Dashboard

Statistics

Patients

Antibiotics

Bacterias

Departments

Cultures

Users

Log Out

Dashboard / Activity Log

Activity Log

For log list

Log Datatable

ID	Name	Email	Description	Date Time
1	admin@rca.com	admin@rca.com	has log in	Thu, Jun 15, 2023 5:58 AM
2	Admin	admin@rca.com	has logged out	Thu, Jun 15, 2023 5:56 AM
3	Admin	admin@rca.com	has created new culture [14]	Thu, Jun 15, 2023 5:24 AM
4	Admin	admin@rca.com	has created new department [9]	Thu, Jun 15, 2023 5:24 AM
5	Admin	admin@rca.com	has created new bacteria group [10]	Thu, Jun 15, 2023 5:24 AM
6	Admin	admin@rca.com	has created new bacteria [139]	Thu, Jun 15, 2023 5:23 AM
7	Admin	admin@rca.com	has created new antibiotic group [11]	Thu, Jun 15, 2023 5:23 AM
8	Admin	admin@rca.com	has created new antibiotic [63]	Thu, Jun 15, 2023 5:23 AM
9	admin@rca.com	admin@rca.com	has log in	Thu, Jun 15, 2023 5:20 AM
10	Mido Said	mido@rca.com	has logged out	Thu, Jun 15, 2023 5:20 AM
11	Mido Said	mido@rca.com	has added a new patient [60]	Thu, Jun 15, 2023 5:20 AM
12	mido@rca.com	mido@rca.com	has log in	Thu, Jun 15, 2023 5:19 AM

This Show Activity Of All Users On System.

RCA
System

mido@rca.com

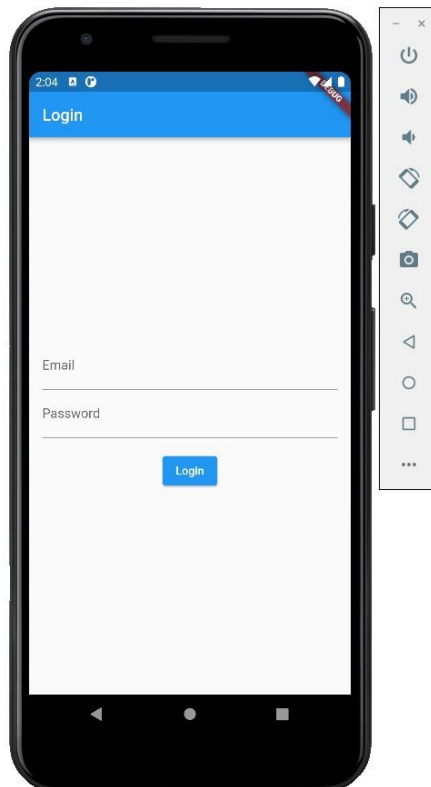
☐ Keep me logged in

Log in

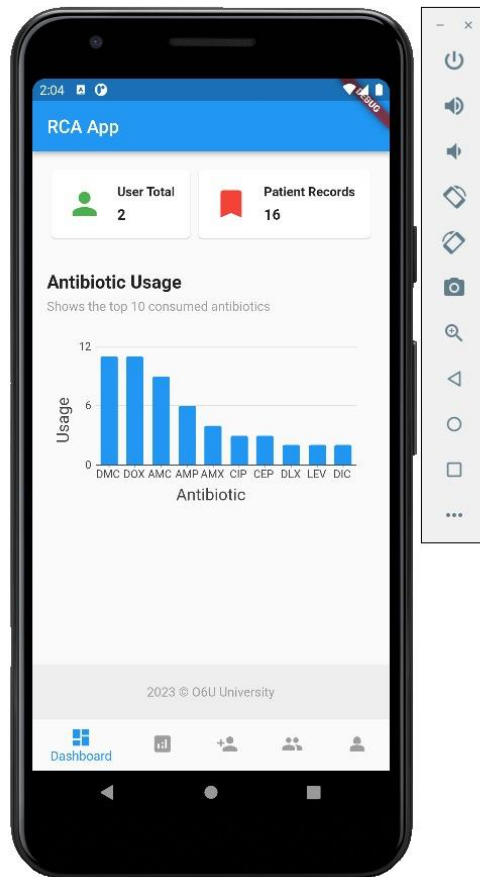


This Show Final Process on System Logging Out of System

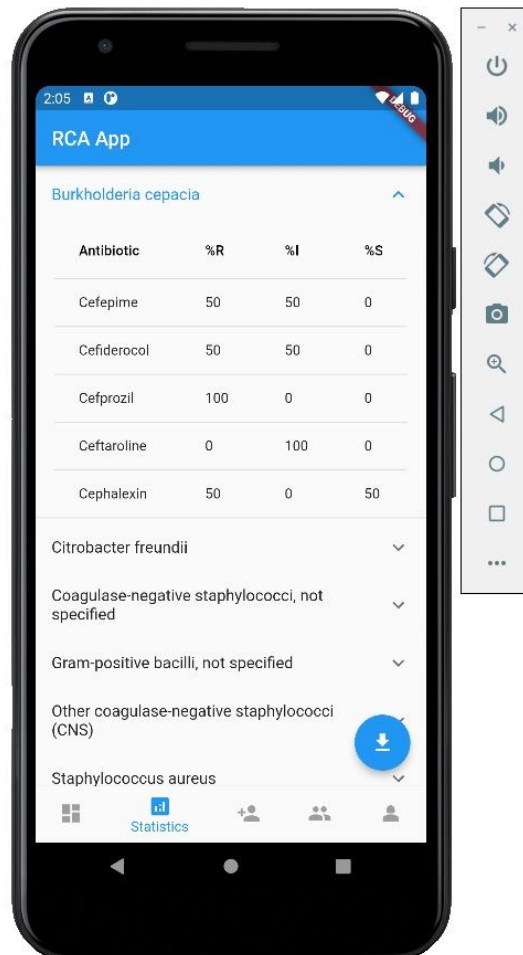
Screen Shoots Of Mobile Application



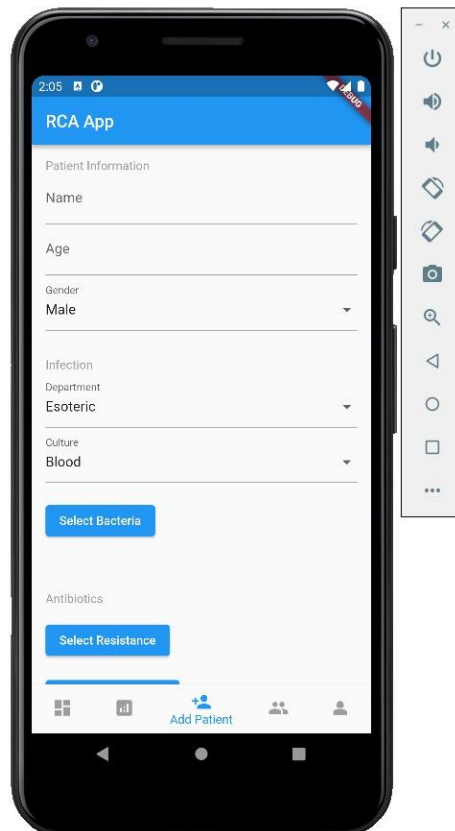
Loge In Page On Mobile Application



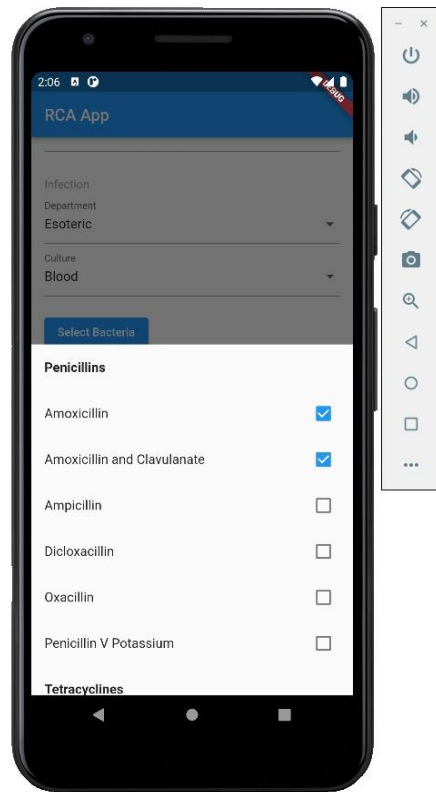
Dashboard Of Mobile Application Show Top 10 Consumed Antibiotics



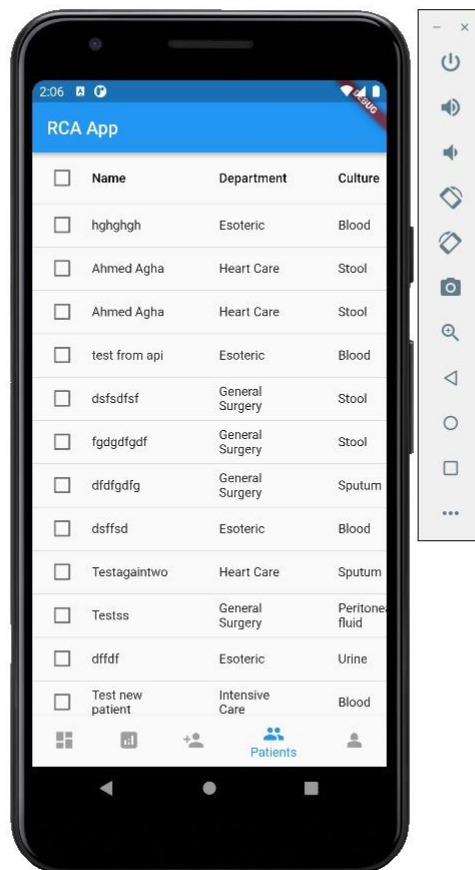
This Page Show Statistics Of Consumption



This Page Show Process Of Adding New Patient On The System



This Page Show Antibiotics



This Page Show Records Of All Patient On The System

Chapter Five

Conclusion & Future Work

5.1 Conclusion:

1. Admin can create many users with different permissions.
2. Users can Fill & Update patient's forms.
3. Admin or Authorized users can monitor consumption statistics.
4. Admin or Authorized users can send system announcements.
5. Admin can track user's activities.
6. Admin can insert/update system data (Departments, Bacteria Types, Antibiotic Types).

5.2 Future Work:

future of rationalizing the consumption of antibiotics will likely involve a combination of technological advancements, increased public education, regulatory oversight, and collaborative efforts among stakeholders. By working together to address the problem of antibiotic resistance, we can help ensure that antibiotics remain effective in treating serious infections and protecting public health for generations to come.

- Increased global collaboration.
- Improved diagnostic tools.
- Development of new antibiotics.
- Develop IOS version from our project.
- Machine learning is booming in these days, there is an intent to use it for improving System to be more efficient and effective.

Appendices:

Front End Code:

RCA systems

System consists of 2 core components, first component is a backend system built with **Laravel PHP** with the help of **Laravel Eloquent** to create, migrate and manipulate database queries.

The first component is responsible for storing all data provided by the users and provide all the data needed for the users.

It's also responsible for handling all type of requests done on the system.

The first component also controlled by an admin dashboard website to control all and track all resources on the system.

The second component is a **MOBILE APPLICATION** built with **Flutter** and it consumes **API** from the backend system.

Frontend tools

Our frontend code is written in **HTML**, **JAVASCRIPT** and **BOOTSTRAP** to develop the frontend of the admin dashboard website.

Also we used **FLUTTER widgets** framework as a frontend in developing the mobile application.

Backend tools

We used **Laravel PHP** to develop and build the system's models, controllers and interfaces.

With the help of **Laravel Auth** to secure and authenticate users and their requests.

Backend Models

5. Patient

In summary, the “Patient” model class represents a patient entity in the system and provides methods to interact with related data, such as departments, cultures, bacteria, users, and antibiotics. It utilizes Laravel’s Eloquent ORM to define relationships and perform database queries.

```
Class Patient extends Model
{
    use HasFactory;

    protected $fillable = [
        'name',
        'age',
        'gender',
        'department_id',
        'culture_id',
        'bacteria_id',
        'user_id'
    ];

    public function department()
    {
        return $this->belongsTo(Department::class);
    }

    public function culture()
    {
        return $this->belongsTo(Culture::class);
    }

    public function bacteria()
    {
        return $this->belongsTo(Bacteria::class);
    }

    public function user()
    {
        return $this->belongsTo(User::class);
    }

    public function antibiotics()
    {
        return $this->belongsToMany(Antibiotic::class)
```

```
        ->withPivot('resistance', 'intermediate', 'sensitive');  
    }
```

```
public function resistance()  
{  
    return DB::table('antibiotic_patient')->where('patient_id', $this->id)  
        ->where('resistance_level', 'resistance')  
        ->pluck('antibiotic_id')->toArray();  
}  
public function intermediate()  
{  
    return DB::table('antibiotic_patient')->where('patient_id', $this->id)  
        ->where('resistance_level', 'intermediate')  
        ->pluck('antibiotic_id')->toArray();  
}  
  
public function sensitive()  
{  
    return DB::table('antibiotic_patient')->where('patient_id', $this->id)  
        ->where('resistance_level', 'sensitive')  
        ->pluck('antibiotic_id')->toArray();  
}  
}
```

2. Antibiotic Model

The “Antibiotic” model class represents an antibiotic entity in the system. It provides relationships with an antibiotic group, patients, and a user. Additionally, it defines an accessor to retrieve the name of the antibiotic group and a custom query method for fetching a data table of antibiotics based on patient counts.

```
Class Antibiotic extends Model
{
    use HasFactory;

    protected $fillable = ['name', 'abbreviation', 'antibiotic_group_id',
'user_id'];

    protected $appends = ['group_name'];

    public function getGroupNameAttribute()
    {
        return $this->antibioticGroup->name;
    }

    public function antibioticGroup()
    {
        return $this->belongsTo(AntibioticGroup::class);
    }

    public function patients()
    {
        return $this->belongsToMany(Patient::class)
            ->withPivot('resistance_level');
    }

    public function user()
    {
        return $this->belongsTo(User::class);
    }

    public function datatable()
    {
        $antibiotics = Antibiotic::withCount('patients')
            ->orderBy('patients_count', 'desc')
            ->limit(10)
    }
}
```

```

        ->get();
        return $antibiotics;
    }
}

```

3. Bacteria Model

In summary, the “Bacteria” model class represents a bacteria entity in the system. It provides relationships with a bacteria group, patients, and a user. Additionally, it defines an accessor to retrieve the name of the bacteria group.

```

Class Bacteria extends Model
{
    use HasFactory;
    protected $fillable = [
        'name',
        'user_id',
        'bacteria_group_id'
    ];

    protected $appends = ['group_name'];

    public function getGroupNameAttribute()
    {
        return $this->group->name;
    }

    public function group()
    {
        return $this->belongsTo(BacteriaGroup::class);
    }

    public function patients()
    {
        return $this->hasMany(Patient::class);
    }

    public function user()
    {
        return $this->belongsTo(User::class);
    }
}

```

4. Department Model

In summary, the “Department” model class represents a department entity in the system. It provides relationships with patients and a user. The model allows you to retrieve the patients associated with a department and determine the user who created the department.

```
Class Department extends Model
{
    protected $fillable = ['name', 'user_id'];

    public function patients()
    {
        return $this->hasMany(Patient::class);
    }

    public function user()
    {
        return $this->belongsTo(User::class);
    }
}
```

5. Culture Model

In summary, the “Culture” model class represents a culture entity in the system. It provides relationships with patients and a user. The model allows you to retrieve the patients associated with a culture and determine the user who created the culture.

```
class Culture extends Model
{
    protected $fillable = ['name', 'user_id'];

    public function patients()
    {
        return $this->hasMany(Patient::class);
    }

    public function user()
    {
        return $this->belongsTo(User::class);
    }
}
```

6. User Model

In summary, the “User” model class represents a user entity in the system, with authentication capabilities. It provides a relationship with a role and defines fillable attributes, hidden attributes, and cast attributes. Each role has different permissions starting from Super admin to normal user. Where super admin has all permissions on the system and normal user has only the permission to insert patients.

```
class User extends Authenticatable
{
    use HasFactory, Notifiable;
    use LockableTrait;

    protected $fillable = [
        'name',
        'email',
        'role_name',
        'password',
    ];

    protected $hidden = [
        'password',
        'remember_token',
    ];

    protected $casts = [
        'email_verified_at' => 'datetime',
    ];

    public function role(): BelongsTo
    {
        return $this->belongsTo(Role::class);
    }
}
```

System Controllers

In summary provide insert, update and delete functionality to all system resources such as users, patients, antibiotics, bacteria and statistics data.

1.Patient Controller

```
class PatientController extends Controller
{
    public function index(Request $request)
    {
        $patients = Patient::with(['department', 'culture', 'bacteria'])
            ->orderBy('created_at', 'desc');

        // Filter by search query
        if ($request->has('search')) {
            $searchTerm = $request->input('search');
            $patients->where(function ($query) use ($searchTerm) {
                $query->where('name', 'LIKE', '%' . $searchTerm . '%');
            });
        }

        $patients = $patients->paginate(10);
        return view('patients.index', compact('patients'));
    }

    public function create()
    {
        // Retrieve all necessary data for the form
        $departments = Department::all();
        $cultures = Culture::all();
        $bacteriaGroups = BacteriaGroup::all();
        $antibioticGroups = AntibioticGroup::all();

        return view(
            'patients.create',
            compact(
                'departments',
                'cultures',
                'bacteriaGroups',
                'antibioticGroups'
            )
        );
    }

    public function store(Request $request)
```

```

{
    $validatedData = $request->validate([
        'name' => 'required',
        'age' => 'required|integer',
        'gender' => 'required',
        'department_id' => 'required|exists:departments,id',
        'culture_id' => 'required|exists:cultures,id',
        'bacteria_id' => 'required|exists:bacterias,id',
    ]);

    // Create the patient
    $patient = new Patient();
    $patient->name = $request->name;
    $patient->age = $request->age;
    $patient->gender = $request->gender;
    $patient->department_id = $request->department_id;
    $patient->culture_id = $request->culture_id;
    $patient->bacteria_id = $request->bacteria_id;
    $patient->user_id = Auth::user()->id;
    $patient->save();

    // Attach the antibiotics with their respective resistances,
    intermediate and sensitivities
    $antibioticData = array();

    foreach ($request->resistance as $r) {
        //
        $antibioticResistanceData[(int)$antibioticId]['resistance_level'] =
        'resistance';
        $antibioticIds = explode(',', $r);
        if ($antibioticIds[0] != "") {
            foreach ($antibioticIds as $id) {
                $antibioticData[] = [
                    'antibiotic_id' => (int) $id, 'resistance_level' =>
                    'resistance'
                ];
            }
        }
    }

    foreach ($request->intermediate as $i) {
        $antibioticIds = explode(',', $i);
        if ($antibioticIds[0] != "") {
            foreach ($antibioticIds as $id) {
                $antibioticData[] = [

```

```

        'antibiotic_id' => (int)$id, 'resistance_level' =>
'intermediate'
    ];
    }
}

foreach ($request->sensitive as $s) {
    $antibioticIds = explode(',', $s);
    if ($antibioticIds[0] != "") {
        foreach ($antibioticIds as $id) {
            $antibioticData[] = [
                'antibiotic_id' => (int)$id, 'resistance_level' =>
'sensitive'
            ];
        }
    }
}

$patient->antibiotics()->sync($antibioticData);

$dt      = Carbon::now();
$todayDate = $dt->toDayDateTimeString();
$activityLog = [
    'name'      => Auth::user()->name,
    'email'     => Auth::user()->email,
    'description' => 'has added a new patient [' . $patient->id . ']',
    'date_time'  => $todayDate,
    'created_at' => Carbon::now(),
];

DB::table('activity_logs')->insert($activityLog);
Toastr::success('Patient has been created successfully!', 'Success');

return redirect()->route('patients.index');
}

public function show(Patient $patient)
{
    return view('patients.show', compact('patient'));
}

public function edit(Patient $patient)
{
    // Retrieve all necessary data for the form
    $departments = Department::all();
    $cultures = Culture::all();
    $bacteriaGroups = BacteriaGroup::all();

```

```

        $antibioticGroups = AntibioticGroup::all();

        return view(
            'patients.edit',
            compact('patient', 'departments', 'cultures', 'bacteriaGroups',
'antibioticGroups')
        );
    }

    public function update(Request $request, Patient $patient)
    {
        // Validate the input data
        $validatedData = $request->validate([
            'name' => 'required',
            'age' => 'required|integer',
            'gender' => 'required',
            'department_id' => 'required|exists:departments,id',
            'culture_id' => 'required|exists:cultures,id',
            'bacteria_id' => 'required|exists:bacterias,id',
        ]);

        // Update the patient
        $patient->name = $request->name;
        $patient->age = $request->age;
        $patient->gender = $request->gender;
        $patient->department_id = $request->department_id;
        $patient->culture_id = $request->culture_id;
        $patient->bacteria_id = $request->bacteria_id;
        $patient->update();

        $patient->antibiotics()->detach();

        $antibioticData = array();

        foreach ($request->resistance as $r) {
            $antibioticIds = explode(',', $r);
            if ($antibioticIds[0] != "") {

                foreach ($antibioticIds as $id) {
                    $antibioticData[] = ['antibiotic_id' => (int)$id,
'resistance_level' => 'resistance'];
                }
            }
        }

        foreach ($request->intermediate as $i) {
            $antibioticIds = explode(',', $i);
            if ($antibioticIds[0] != "") {

```

```

        foreach ($antibioticIds as $id) {
            $antibioticData[] = ['antibiotic_id' => (int)$id,
'resistance_level' => 'intermediate'];
        }
    }

    foreach ($request->sensitive as $s) {
        $antibioticIds = explode(',', $s);
        if ($antibioticIds[0] != "") {
            foreach ($antibioticIds as $id) {
                $antibioticData[] = ['antibiotic_id' => (int)$id,
'resistance_level' => 'sensitive'];
            }
        }
    }

    $patient->antibiotics()->sync($antibioticData);

    $dt          = Carbon::now();
    $todayDate   = $dt->toDayDateTimeString();
    $activityLog = [
        'name'      => Auth::user()->name,
        'email'     => Auth::user()->email,
        'description' => 'has updated patient [' . $patient->id . ']'
info',
        'date_time' => $todayDate,
        'created_at' => Carbon::now(),
    ];
    DB::table('activity_logs')->insert($activityLog);
    Toastr::success('Patient has been updated successfully!', 'Success');
    // Redirect to the index page with a success message
    return redirect()->route('patients.index');
}

public function destroy(Patient $patient)
{
    // Delete the patient and detach all the associated antibiotics
    $patient->antibiotics()->detach();
    $patient->delete();

    // Redirect to the index page with a success message
    return redirect()->route('patients.index')->with('success', 'Patient
has been deleted successfully!');
}

```

```

public function datatable(Patient $patient)
{
    return Patient::all()->toArray();
}

public function APIcreate(Request $request)
{
    $validatedData = $request->validate([
        'name' => 'required',
        'age' => 'required|integer',
        'gender' => 'required',
        'departmentId' => 'required|exists:departments,id',
        'cultureId' => 'required|exists:cultures,id',
        'bacteriaId' => 'required|exists:bacterias,id',
    ]);

    // Create the patient
    $patient = new Patient();
    $patient->name = $request->name;
    $patient->age = $request->age;
    $patient->gender = $request->gender;
    $patient->department_id = $request->departmentId;
    $patient->culture_id = $request->cultureId;
    $patient->bacteria_id = $request->bacteriaId;
    $patient->user_id = $request->userId;
    $patient->save();

    // Attach the antibiotics with their respective resistances,
    intermediate and sensitivities
    $antibioticData = array();

    foreach ($request->resistanceIds as $r) {
        //
        $antibioticResistanceData[(int)$antibioticId]['resistance_level'] =
        'resistance';
        $antibioticIds = explode(',', $r);
        if ($antibioticIds[0] != "") {
            foreach ($antibioticIds as $id) {
                $antibioticData[] = ['antibiotic_id' => (int)$id,
                'resistance_level' => 'resistance'];
            }
        }
    }

    foreach ($request->intermediateIds as $i) {
        $antibioticIds = explode(',', $i);
        if ($antibioticIds[0] != "") {

```

```

        foreach ($antibioticIds as $id) {
            $antibioticData[] = ['antibiotic_id' => (int)$id,
'resistance_level' => 'intermediate'];
        }
    }

    foreach ($request->sensitiveIds as $s) {
        $antibioticIds = explode(',', $s);
        if ($antibioticIds[0] != "") {
            foreach ($antibioticIds as $id) {
                $antibioticData[] = ['antibiotic_id' => (int)$id,
'resistance_level' => 'sensitive'];
            }
        }
    }

    $patient->antibiotics()->sync($antibioticData);

    $dt          = Carbon::now();
    $user = User::find($request->userId);
    $todayDate   = $dt->toDayDateTimeString();
    $activityLog = [
        'name'          => $user->name,
        'email'         => $user->email,
        'description'    => 'has added a new patient [' . $patient->id . ']',
        'date_time'     => $todayDate,
        'created_at'    => Carbon::now(),
    ];

    DB::table('activity_logs')->insert($activityLog);

    return response('created', 201);
}

public function IndexForApp(Request $request)
{
    $patients = Patient::with(['department', 'culture', 'bacteria'])->
        orderBy('created_at', 'desc');

    // Filter by search query
    if ($request->has('search')) {
        $searchTerm = $request->input('search');
        $patients->where(function ($query) use ($searchTerm) {
            $query->where('name', 'LIKE', '%' . $searchTerm . '%');
        });
    }
}

```

```

        $patients = $patients->get();
        return $patients;
    }

    function patientsRecordsForApp()
    {
        $patientRecords = Patient::all()->count();
        $response = array('patientRecords' => $patientRecords);
        return json_encode($response);
    }
}

```

2. Antibiotic Controller

```

class AntibioticController extends Controller
{
    /**
     * Display a listing of the resource.
     *
     * @return \Illuminate\Http\Response
     */
    public function index()
    {
        $antibiotics = Antibiotic::all();
        return view('antibiotics.index', compact(['antibiotics']));
    }

    /**
     * Show the form for creating a new resource.
     *
     * @return \Illuminate\Http\Response
     */
    public function create()
    {
        $antibioticGroups = AntibioticGroup::all();
        return view('antibiotics.create', compact(['antibioticGroups']));
    }

    /**
     * Store a newly created resource in storage.
     *
     * @param \Illuminate\Http\Request $request
     * @return \Illuminate\Http\Response
     */
}

```

```

public function store(Request $request)
{
    // Validate the input data
    $validatedData = $request->validate([
        'name' => 'required|unique:antibiotics',
        'abbreviation' => 'required',
        'antibiotic_group_id' => 'required|exists:antibiotic_groups,id',
    ]);

    $antibiotic = Antibiotic::create(['user_id' => Auth::user()->id,
...$validatedData]);

    $dt          = Carbon::now();
    $todayDate   = $dt->toDayDateTimeString();
    $activityLog = [
        'name'      => Auth::user()->name,
        'email'     => Auth::user()->email,
        'description' => 'has created new antibiotic [' . $antibiotic->id
. ']',
        'date_time'  => $todayDate,
        'created_at' => Carbon::now(),
    ];
    DB::table('activity_logs')->insert($activityLog);
    Toastr::success('Antibiotic has been created successfully!',
'Success');

    return redirect()->route('antibiotics.index');
}

/**
 * Display the specified resource.
 *
 * @param int $id
 * @return \Illuminate\Http\Response
 */
public function show($id)
{
    //
}

/**
 * Show the form for editing the specified resource.
 *
 * @param int $id
 * @return \Illuminate\Http\Response
 */
public function edit($id)

```

```

{
    $antibiotic = Antibiotic::find($id);
    $antibioticGroups = AntibioticGroup::all();
    return view('antibiotics.edit', compact(['antibiotic',
'antibioticGroups']));
}

/**
 * Update the specified resource in storage.
 *
 * @param \Illuminate\Http\Request $request
 * @param int $id
 * @return \Illuminate\Http\Response
 */
public function update(Request $request, $id)
{
    // Validate the input data
    $validatedData = $request->validate([
        'name' => 'required|unique:antibiotics',
        'abbreviation' => 'required',
        'antibiotic_group_id' => 'required|exists:antibiotic_groups,id',
    ]);

    $antibiotic = Antibiotic::find($id);
    $antibiotic->name = $request->name;
    $antibiotic->antibiotic_group_id = $request->antibiotic_group_id;
    $antibiotic->update();

    $dt = Carbon::now();
    $todayDate = $dt->toDayDateTimeString();
    $activityLog = [
        'name' => Auth::user()->name,
        'email' => Auth::user()->email,
        'description' => 'has updated antibiotic [' . $antibiotic->id . ']'
info',
        'date_time' => $todayDate,
        'created_at' => Carbon::now(),
    ];
    DB::table('activity_logs')->insert($activityLog);
    Toastr::success('Antibiotic has been updated successfully!',
'Success');

    return redirect()->route('antibiotics.index');
}

/**
 * Remove the specified resource from storage.
 *

```

```

    * @param int $id
    * @return \Illuminate\Http\Response
    */
    public function destroy($id)
    {
        $antibiotic = Antibiotic::find($id);
        $antibiotic->delete();
        // Redirect to the index page with a success message
        Toastr::success('Antibiotic has been deleted successfully!',
'Success');
        return redirect()->route('antibiotics.index');
    }

    public function datatable()
    {
        $antibiotics = Antibiotic::withCount('patients')
            ->orderBy('patients_count', 'desc')
            ->limit(10)
            ->get();

        return $antibiotics;
    }

    public function all()
    {
        return Antibiotic::all();
    }

    function antibioticUsageForApp()
    {
        $antibiotics = Antibiotic::select('abbreviation')
            ->withCount('patients')
            ->orderByDesc('patients_count')
            ->limit(10)
            ->get()
            ->map(function ($antibiotic) {
                return [
                    'name' => $antibiotic->abbreviation,
                    'usage' => $antibiotic->patients_count
                ];
            })
            ->toArray();
        $response = array('antibioticUsage' => $antibiotics);
        return json_encode($response);
    }
}

```

3. Statistics Controller

```
class StatisticsController extends Controller
{
    //
    public function index(Request $request)
    {
        // Retrieve all departments, Bacteria groups
        $departments = Department::all();
        $bacteriaGroups = BacteriaGroup::all();

        // Retrieve filter string parameters
        $startDate = $request->query('date_from');
        $endDate = $request->query('date_to');
        $departmentId = $request->query('department');
        $gender = $request->query('gender');
        $bacteria = $request->query('bacteria');

        // Initialize query builder for statistics
        $bacteriaStatistics = Bacteria::select('bacterias.name as
bacteria_name', 'antibiotics.name as antibiotic_name', DB::raw('count(case
when antibiotic_patient.resistance_level = "resistance" then 1 end) as
resistance_count'), DB::raw('count(case when
antibiotic_patient.resistance_level = "intermediate" then 1 end) as
intermediate_count'), DB::raw('count(case when
antibiotic_patient.resistance_level = "sensitive" then 1 end) as
sensitive_count'))
        ->join('patients', 'patients.bacteria_id', '=', 'bacterias.id')
        ->join('antibiotic_patient', 'antibiotic_patient.patient_id', '=',
'patients.id')
        ->join('antibiotics', 'antibiotics.id', '=',
'antibiotic_patient.antibiotic_id')
        ->groupBy('bacterias.name', 'antibiotics.name');

        // Apply filters to query builder
        if ($departmentId) {
            $bacteriaStatistics->where('department_id', '=', $departmentId);
        }

        if ($gender) {
            $bacteriaStatistics->where('gender', '=', $gender);
        }

        if ($startDate) {
            $bacteriaStatistics->where(
```

```

        'patients.created_at',
        '>=',
        $startDate
    );
}

if ($endDate) {
    $bacteriaStatistics->where(
        'patients.created_at',
        '<=',
        $endDate
    );
}

if ($bacteria) {
    $bacteriaStatistics->where('bacteria_id', '=', $bacteria);
}

// Execute query builder and retrieve statistics
$bacteriaStatistics = $bacteriaStatistics->get();

$emptyMessage = null;
if (count($bacteriaStatistics) == 0) {
    $emptyMessage = 'No Data found for your filter';
}

// Group the statistics by bacteria name
$groupedStatistics = $bacteriaStatistics->groupBy('bacteria_name');

// Convert the grouped statistics into an array of arrays
$statisticsArray = [];
foreach ($groupedStatistics as $bacteriaName => $statistics) {
    $antibioticStatistics = [];
    foreach ($statistics as $stat) {
        $total = $stat->resistance_count + $stat->intermediate_count +
        $stat->sensitive_count;
        $resistancePercent = $total > 0 ? round($stat->
        resistance_count / $total * 100, 2) : 0;
        $intermediatePercent = $total > 0 ? round($stat->
        intermediate_count / $total * 100, 2) : 0;
        $sensitivePercent = $total > 0 ? round($stat->sensitive_count
        / $total * 100, 2) : 0;
        $antibioticStatistics[] = [
            'antibiotic_name' => $stat->antibiotic_name,
            'resistance_percent' => $resistancePercent,
            'intermediate_percent' => $intermediatePercent,
            'sensitive_percent' => $sensitivePercent,

```

```

    ];
}
$statisticsArray[] = [
    'bacteria_name' => $bacteriaName,
    'antibiotic_statistics' => $antibioticStatistics,
];
}

// Sort the statistics array by bacteria name
usort($statisticsArray, function ($a, $b) {
    return strcmp($a['bacteria_name'], $b['bacteria_name']);
});

// return $statisticsArray;
return view('statistics/index', compact(['statisticsArray',
'departments', 'bacteriaGroups', 'emptyMessage']));
}
public function indexForApp(Request $request)
{
    // Retrieve all departments, Bacteria groups
    $departments = Department::all();
    $bacteriaGroups = BacteriaGroup::all();

    // Retrieve filter string parameters
    $startDate = $request->query('date_from');
    $endDate = $request->query('date_to');
    $departmentId = $request->query('department');
    $gender = $request->query('gender');
    $bacteria = $request->query('bacteria');

    // Initialize query builder for statistics
    $bacteriaStatistics = Bacteria::select('bacterias.name as
bacteria_name', 'antibiotics.name as antibiotic_name', DB::raw('count(case
when antibiotic_patient.resistance_level = "resistance" then 1 end) as
resistance_count'), DB::raw('count(case when
antibiotic_patient.resistance_level = "intermediate" then 1 end) as
intermediate_count'), DB::raw('count(case when
antibiotic_patient.resistance_level = "sensitive" then 1 end) as
sensitive_count'))
->join('patients', 'patients.bacteria_id', '=', 'bacterias.id')
->join('antibiotic_patient', 'antibiotic_patient.patient_id', '=',
'patients.id')
->join('antibiotics', 'antibiotics.id', '=',
'antibiotic_patient.antibiotic_id')
->groupBy('bacterias.name', 'antibiotics.name');

    // Apply filters to query builder
    if ($departmentId) {

```

```

        $bacteriaStatistics->where('department_id', '=', $departmentId);
    }

    if ($gender) {
        $bacteriaStatistics->where('gender', '=', $gender);
    }

    if ($startDate) {
        $bacteriaStatistics->where(
            'patients.created_at',
            '>=',
            $startDate
        );
    }

    if ($endDate) {
        $bacteriaStatistics->where(
            'patients.created_at',
            '<=',
            $endDate
        );
    }

    if ($bacteria) {
        $bacteriaStatistics->where('bacteria_id', '=', $bacteria);
    }

    // Execute query builder and retrieve statistics
    $bacteriaStatistics = $bacteriaStatistics->get();

    $emptyMessage = null;
    if (count($bacteriaStatistics) == 0) {
        $emptyMessage = 'No Data found for your filter';
    }

    // Group the statistics by bacteria name
    $groupedStatistics = $bacteriaStatistics->groupBy('bacteria_name');

    // Convert the grouped statistics into an array of arrays
    $statisticsArray = [];
    foreach ($groupedStatistics as $bacteriaName => $statistics) {
        $antibioticStatistics = [];
        foreach ($statistics as $stat) {
            $total = $stat->resistance_count + $stat->intermediate_count +
            $stat->sensitive_count;
            $resistancePercent = $total > 0 ? round($stat->
            >resistance_count / $total * 100, 2) : 0;

```

```
        $intermediatePercent = $total > 0 ? round($stat->intermediate_count / $total * 100, 2) : 0;
```

```
        $sensitivePercent = $total > 0 ? round($stat->sensitive_count / $total * 100, 2) : 0;
        $antibioticStatistics[] = [
            'antibiotic_name' => $stat->antibiotic_name,
            'resistance_percent' => $resistancePercent,
            'intermediate_percent' => $intermediatePercent,
            'sensitive_percent' => $sensitivePercent,
        ];
    }
    $statisticsArray[] = [
        'bacteria_name' => $bacteriaName,
        'antibiotic_statistics' => $antibioticStatistics,
    ];
}

// Sort the statistics array by bacteria name
usort($statisticsArray, function ($a, $b) {
    return strcmp($a['bacteria_name'], $b['bacteria_name']);
});
return $statisticsArray;
}

public function exportPDF(Request $request)
{
    $statisticsArray = json_decode(urldecode($request->statisticsArray), true);

    $pdf = app('dompdf.wrapper')->loadView('statistics.export', compact('statisticsArray'));

    return $pdf->download('antibiotic-statistics.pdf');
}
}
```

System API

These routes define various endpoints for different functionalities of the application, allowing clients to interact with the API and perform actions such as authentication, patient management, antibiotic management, statistics retrieval, and user management.

```
<?php
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Route;

Route::prefix('v1')->group(function () {
    Route::post('/login', [AuthController::class, 'login']);
    Route::post('/logout', [AuthController::class, 'logoutFromApp']);
    Route::get('/patients', [PatientController::class, 'indexForApp']);
    Route::get('/patientsDatatable', [PatientController::class, 'datatable']);
    Route::get('/antibioticsDatatable', [AntibioticController::class,
'datatable']);
    Route::get('/statistics', [StatisticsController::class, 'indexForApp']);
    Route::get('/antibioticGroups', [AntibioticGroupController::class,
'all']);
    Route::get('/antibiotics', [AntibioticController::class, 'all']);
    Route::get('/antibioticDatatable', [AntibioticController::class,
'datatable']);
    Route::get('/departments', [DepartmentController::class, 'all']);
    Route::get('/bacteria', [BacteriaController::class, 'all']);
    Route::get('/bacteriaGroups', [BacteriaGroupController::class, 'all']);
    Route::get('/cultures', [CultureController::class, 'all']);
    Route::post('/createPatient', [PatientController::class, 'APIcreate']);
    Route::get('/totalUsers', [UserManagementController::class,
'totalUsersForApp']);
    Route::get('/patientsRecords', [PatientController::class,
'patientsRecordsForApp']);
    Route::get('/antibioticUsage', [AntibioticController::class,
'antibioticUsageForApp']);
});
```

System Packages and Addons

```
"devDependencies": {
  "axios": "^0.19",
  "bootstrap": "^4.0.0",
  "cross-env": "^7.0",
  "jquery": "^3.2",
  "laravel-mix": "^5.0.1",
  "lodash": "^4.17.19",
  "popper.js": "^1.12",
  "resolve-url-loader": "^2.3.1",
  "sass": "^1.20.1",
  "sass-loader": "^8.0.0",
  "select2": "^4.1.0-rc.0",
  "vue": "^2.5.17",
  "vue-template-compiler": "^2.6.10"
},
"dependencies": {
  "datepicker": "^0.0.0"
}
```

References :

1. PHP-<https://www.php.net/>
2. HTML-<https://html.com/>
3. CSS- <https://developer.mozilla.org/en-US/>
4. Java Script - <https://www.javascript.com/>
5. Bootstrap- <https://getbootstrap.com/>
6. GitHub- <https://github.com/>
7. jQuery- <https://jquery.com/>
8. Laravel- <https://laravel.com/>
9. React- <https://react.dev/>
10. My SQL- <https://www.mysql.com/>
11. Flutter- <https://flutter.dev/>
12. W3 School- <https://www.w3schools.com/>
13. Stack Over Flow- <https://stackoverflow.com/>
14. WHO- <https://whonet.org/>

ملخص المشروع

تعتبر المضادات الحيوية من الأدوية الهامة لعلاج الالتهابات البكتيرية. ومع ذلك ، فقد أدى الإفراط في استخدامها وإساءة استخدامها إلى ظهور البكتيريا المقاومة للمضادات الحيوية ، والتي تشكل تهديدًا كبيرًا للصحة العامة العالمية.

يعد ترشيد استهلاك المضادات الحيوية أمرًا ضروريًا للحفاظ على فعاليتها وضمان توافرها للأجيال القادمة. سنحقق ذلك من خلال نظام التطبيق والموقع الإلكتروني الذي يتضمن تحسين استخدام المضادات الحيوية من خلال ممارسات الوصفات المناسبة والمراقبة وتطوير علاجات بديلة



كلية نظم المعلومات وعلوم الحاسب



جامعة 6 أكتوبر

مشروع التخرج

نظام ترشيد استهلاك المضادات الحيوية

أعد بواسطة

القسم	اسماء الطلاب
علوم حاسب	محمد سعيد عبدالفتاح
علوم حاسب	احمد محمد اسماعيل
علوم حاسب	راضي محمد راضي
نظم معلومات	عرفه احمد سالم
نظم معلومات	علي احمد علي
نظم معلومات	علاء عبدالله توفيق

تحت إشراف

دكتور مهندس/ سامي السيد المقدم الدكتور/ عبدالرحمن الاخضر

الهيئة المعاونة

م. / حمد رشاد

يونيو ٢٠٢٣